

RESEARCH

Open Access



QHRMOF: A Quantum-Inspired hybrid Multi-Objective framework for Energy-Efficient task scheduling and load balancing in cloud computing

Umesh Kumar Lilhore¹, Scaria Alex², Vince Paul³, Rahoof Puthan Purayil⁴, Sultan Mesfer A Aldossary⁵, Sarita Simaiya¹, Ehab seif Ghith⁶, Heba G. Mohamed⁷ and Monish Khan^{8*}

Abstract

The swift expansion of cloud computing services has resulted in a notable increase in energy consumption, presenting challenges for sustainability and effective resource management in cloud data centres. This paper introduces a novel Quantum-Inspired Hybrid Reinforcement Learning and Multi-Objective Optimization Framework (QHRMOF) designed to optimize task scheduling, dynamic load balancing, and server consolidation while minimizing power consumption and enhancing system performance. QHRMOF integrates three fundamental techniques: Quantum-Inspired Evolutionary Algorithm (QIEA) employs quantum principles like superposition and entanglement to improve solution space exploration and mitigate local optima; Hybrid Deep Reinforcement Learning (HDRL) integrates convolutional neural networks (CNNs) and Long Short-Term Memory (LSTM) to forecast workloads and dynamically categorize virtual machines (VMs) into overloaded and underloaded states for efficient task migration and load balancing; and Multi-Objective Optimisation (MOO) reconciles multiple conflicting objectives, including minimizing energy consumption and makespan while maximizing resource utilization and system scalability. QHRMOF minimizes unnecessary migrations and overhead through informed decision-making and adaptive resource management, all while maintaining optimal system performance. Simulations performed on the CloudSim platform utilizing real-world datasets, including those from NASA, HPC2N, and Google workloads, indicate that the proposed framework surpasses leading methodologies such as Multi-objective Genetic Algorithm (MOGA), Particle Swarm Optimization (PSO), Deep Reinforcement Learning for Load Balancing (DRL-LB), and Ant Colony Optimization (ACO). The findings indicate that QHRMOF attains a maximum reduction of 18.76% in makespan, a 22.84% decrease in energy consumption, a 19.52% enhancement in resource utilization, a 25.39% improvement in load balancing efficiency, and a 12.67% reduction in failure rates. These findings confirm the efficacy of QHRMOF in enhancing resource management, augmenting system reliability, and fostering energy-efficient cloud computing operations.

*Correspondence:
Monish Khan
drkumacse@gmail.com

Full list of author information is available at the end of the article

Keywords Cloud computing, Quantum-Inspired evolutionary algorithm, Deep reinforcement learning, Multi-Objective optimization, Task scheduling, Load balancing

Introduction

Cloud computing is a revolutionary technology that enables organizations to utilize a wide range of computational resources and services over the Internet. The Cloud provides computing as a service, which you can scale as needed, using only the resources you need, with prices based on the resources you use (such as servers, storage, applications, etc.). In other words, it offers on-demand access to a shared pool of computing resources. Cloud services usage continues to explode globally as enterprises pivot to data-intensive applications, AI models, and IoT services in the cloud [1–3].

Cloud computing has introduced significant benefits in terms of innovation and service delivery; however, this rapid growth has also been accompanied by a steep increase in electricity consumption by cloud data centers, which now account for around 1% of the total global electricity [4]. This increased energy usage not only raises cloud providers' operational costs but also contributes to environmental issues, resulting in higher carbon emissions. As a result, improving the energy efficiency of cloud data centers with acceptable performance has attracted considerable attention in research [5, 6].

Challenges in energy efficiency, resource management, and task scheduling

Cloud data centers comprise numerous physical servers that host multiple virtual machines (VMs), managing various and dynamic workloads generated by end-users. Managing these VMs while ensuring optimal task scheduling, dynamic load balancing, and server consolidation presents numerous challenges that affect overall system performance and energy efficiency [3–6].

- *Task Scheduling Complexity:* Task scheduling is a critical component of cloud computing, wherein incoming user requests (tasks) are allocated to available resources to minimize the makespan, decrease power consumption, and optimize resource utilization. Conventional task scheduling algorithms often employ static methods that fail to adapt to varying workloads, resulting in inefficient task allocation. The potential consequences of inadequate task scheduling, including extended project timelines, increased energy consumption, and higher costs, underscore the necessity of more intelligent and adaptable scheduling systems [3].
- *Dynamic Load Balancing Issues:* In cloud environments, load balancing distributes workloads evenly across multiple servers to prevent resource

overloading or underutilization. An imbalance in load distribution can lead to server overloading, which degrades system performance and increases energy consumption or underutilization, resulting in wasted resources. Inefficient resource utilization and poor system responsiveness frequently result from traditional load-balancing techniques that fail to account for dynamic workload variations. Ensuring optimal system efficiency requires the deployment of real-time workload forecasting and virtual machine categorization to facilitate effective load balancing [4].

- *Server Consolidation and Energy Efficiency:* Server consolidation involves identifying underutilized servers and migrating VMs to minimize the number of active servers, thereby reducing overall power consumption. Excessive virtual machine (VM) migrations can create additional overhead and increase the risk of violating service-level agreements (SLAs). Achieving a balance between energy efficiency and performance requires astute consolidation methods that minimize unnecessary migrations while ensuring optimal resource utilization [5].
- *Resource Management and Scalability:* As cloud environments become increasingly complex and heterogeneous, effectively managing resources while maintaining scalability presents a significant challenge. Resource management strategies must adapt to changing workloads, scale flexibly, and ensure reliability while maintaining performance [6–8].

Motivation of the research

Considering these challenges, there is an increasing demand for creative solutions that can flexibly adapt to changes in workload, optimize resource utilization, and enhance energy efficiency in cloud environments. Recent advancements in artificial intelligence, particularly in reinforcement learning and quantum-inspired optimization techniques, have yielded encouraging results in addressing complex decision-making challenges [7–10].

- *Potential of QIEA:* The QIEA mimics the principles of quantum mechanics, such as superposition and entanglement, to explore large solution spaces more efficiently than traditional evolutionary algorithms. QIEA has demonstrated an exceptional ability to avoid local optima and discover globally optimal solutions within high-dimensional search spaces.

Regarding task scheduling and server consolidation, QIEA can enhance the quality of solutions by intelligently examining various configurations and reducing overall energy consumption [7].

- *Advancements in DRL:* The capacity of reinforcement learning and neural networks is combined to facilitate adaptive learning and intelligent decision-making in DRL. DRL can make real-time decisions regarding task scheduling and VM migration, dynamically classify VMs into overloaded and underloaded categories, and accurately predict workload patterns using CNNs and LSTM models. This predictive capability contributes to the optimization of load balancing, reduces unnecessary migrations, and improves overall system efficiency [8].
- *Motivation for Hybrid Framework:* A hybrid approach that combines the strengths of QIEA and DRL can address the shortcomings of traditional task scheduling, load balancing, and server consolidation techniques. QIEA offers enhanced exploration capabilities for task scheduling, while DRL improves the framework's adaptability by dynamically responding to workload variations and executing intelligent load-balancing decisions. Integrating these approaches into a cohesive framework enables a balance between exploration and exploitation, resulting in improved energy efficiency, reduced makespan, and increased resource utilization [9].

Mapping existing gaps to the proposed solution

Despite advancements in cloud computing, task scheduling, load balancing, and server consolidation methods, these methods have still not been able to cater to all the properties of the dynamic modern cloud environment. More precisely, current solutions face the following challenges that need to be tackled for improving efficiency, minimizing energy consumption, and optimizing resource usage:

- *Static Task Scheduling:* Average task scheduling algorithms rely on static or heuristic methods that fail in dynamic environments, and workload pattern changes rapidly. This rigidity affects their functional adaptation to workload variations, and subsequent task allocation using these static methods leads to poor resource utilization and (results in back-to-back extensions to their makespan also.) Moreover, task scheduling either employs static rule-based or optimization techniques, which fail to meet the dynamic nature of resource demands and thus do not provide overall good energy efficiency [10–12].
- *Excessive Virtual Machine (VM) Migrations:* Many modern systems use a VM migration strategy that

is simple-minded and does not consider whether or not VM migration is warranted. Although load balancing migrations serve this purpose, frequent migrations often create additional overhead and generate heavier network traffic, as well as increased energy consumption. Evictions can be even worse in cases where unnecessary VM migrations are initiated as a result of incorrect prediction of future workloads or insufficient VM classification (overloaded vs. underloaded states) [13, 14].

- *Inadequate Handling of Multiple Conflicting Objectives:* Existing approaches typically give priority to a specific optimization objective, e.g., makespan and energy consumption, while ignoring others like resource utilization, scalability of the system, etc. This one-dimensional policy renders them ineffective in addressing the conflicting objectives common in traditional cloud resource management, where energy efficiency, task completion time, and resource allocation must be fulfilled simultaneously, with the trade-off among each pair of objectives needing to be balanced [15]–[16].
- *Poor Adaptability to Dynamic Workloads:* Traditional systems mostly lack support for dynamic workload prediction decision-making. The lack of real-time forecasting of workload patterns means that these systems cannot quickly adjust to changes in demand or resource availability, thereby undermining system performance and leading to energy wastage [17].

QHRMOF hybrid approach: unique solution to existing gaps

A Quantum-Inspired Hybrid Reinforcement Learning and Multi-Objective Framework (QHRMOF) has been proposed that overcomes these limitations [5, 18]. Whereas existing quantum-inspired (QI) or RL-based methods have inherent limitations when applied alone, QHRMOF successfully transcends these limitations thanks to its hybrid nature.

- *QIEA's Role in Overcoming Static Scheduling Limitations:* By exploiting quantum properties such as superposition and entanglement to traverse a more extensive solution space more efficiently than classical evolutionary algorithms, QIEA can overcome static scheduling limitations. QIEA has the advantage of dynamic task allocations, unlike static methods, which allow for the intelligent optimization of task-to-resource assignments given the changing nature of typical workloads. This exploration adequacy expands QIEA to escape local optima and adapt to evolving task requirements, thereby achieving optimal energy consumption and makespan [4–6].

- *Adaptable for Real-Time Load Balancing and VM Classification:* With CNNs and LSTMs working for feature extraction and workload prediction, respectively, HDRL is capable of classifying the VMs into overloaded and underloaded in real time and hence enables adaptive load balancing. This ensures that only necessary migrations of VMs are triggered, preventing unnecessary overhead from excessive migrations. In addition, HDRL continuously learns and adapts when the workload changes, providing near real-time, intelligent responses to the fluctuations in task and resource demands, and therefore helps deliver efficient decision-making in dynamic environments [19].
- *MOO's Approach to Multi-Objective Optimization:* Conventional approaches usually focus on single-objective problems, but the MOO strategy in QHRMOF optimizes multiple (or more than two) conflicting goals at the same time. QHRMOF provides a Pareto Front of trade-off solutions, which means the system can balance energy consumption, makespan, and resource utilization without sacrificing one objective for the gain of another. Such multidimensional optimization is crucial for addressing the challenges presented by modern clouds, where energy efficiency and system performance must be balanced simultaneously [3–7, 21].
- *Integrated Task Scheduling and Load Balancing Framework:* QHRMOF provides a holistic and adaptive solution to prevent the independent approach taken in traditional literature on task scheduling, load balancing, and VM migration. The integration of multiple information sources enables the system to analyze both short-term workloads and long-term predictions, making it more flexible in response to dynamic changes. The combined effect of QIEA, HDRL, and MOO enables the system to adapt to workload variations, reduce unwanted migrations, and achieve higher overall performance than state-of-the-art solutions.
- *Development of a QIEA:* QIEA utilizes quantum principles to investigate intricate solution spaces efficiently, optimizing task scheduling while minimizing makespan and power consumption.
- *Integration of HDRL:* A hybrid CNN-LSTM model predicts workload patterns and dynamically classifies VMs into overloaded and underloaded states, enabling intelligent load balancing and task migration.
- *Multi-Objective Optimization for Conflicting Objectives:* A MOO strategy optimizes several conflicting objectives, such as minimizing makespan and energy consumption while maximizing resource utilization and system scalability.
- *Intelligent Server Consolidation Mechanism:* The proposed framework effectively consolidates underutilized servers, minimizing unnecessary VM migrations and thereby enhancing overall system performance while reducing overhead.
- *Extensive Performance Evaluation Using Real-World Datasets:* The effectiveness of the proposed QHRMOF framework is validated through simulation experiments conducted on the CloudSim platform using real-world datasets (NASA, HPC2N, and Google workloads), compared to existing approaches such as MOGA, PSO, DRL-LB, and ACO.

Organization of the Article

The subsequent sections of this article are structured as follows:

- *Sect. 2 (Related Work):* It examines contemporary research on task scheduling, load balancing, and energy efficiency in cloud computing, emphasizing the shortcomings of current methodologies and pinpointing research deficiencies.
- *Sect. 3 (Proposed Framework):* It presents a detailed description of the proposed hybrid framework, including the design of the quantum-inspired evolutionary algorithm, the structure of the hybrid deep reinforcement learning model, and the multi-objective optimization strategy.
- *Sect. 4 (Experimental Setup and Methodology):* This section outlines the experimental environment, datasets, and evaluation metrics used to assess the effectiveness of the proposed framework.
- *Sect. 5 (Results and Discussion):* This section presents the outcomes of the simulation experiments, comparing QHRMOF with cutting-edge

Key contributions and objectives of the proposed framework

This paper proposes a QHRMOF to tackle challenges and harness the capabilities of advanced optimization and machine learning techniques for optimizing task scheduling, dynamic load balancing, and server consolidation in cloud computing environments [11–14]. This research's principal contributions are delineated as follows:

methodologies and evaluating its efficacy in terms of makespan, energy consumption, resource utilization, and scalability.

- *Sect. 6 (Conclusion and Future Work)*: It summarises the principal findings and delineates prospective avenues for future research to augment the efficiency and resilience of cloud computing systems.

Review of existing work

Cloud computing has transformed the way we manage data and applications by providing flexible resources and substantial computational power. However, as the need grows, managing tasks, optimizing energy use, and balancing loads remain significant hurdles. The literature has explored approaches to address these challenges using heuristic, meta-heuristic, and machine-learning methods. This section thoroughly examines task scheduling methodologies, energy-efficient frameworks, and hybrid optimization strategies, accompanied by an analysis of the shortcomings in current solutions and the rationale for the proposed QHRMOF.

Review of task scheduling techniques in cloud computing

Task scheduling is crucial for optimizing the performance and efficiency of cloud systems, as it determines how tasks are allocated to virtual machines (VMs). Traditional algorithms, such as Round Robin (RR), Min-Min, and Max-Min, have been commonly utilized; however, they frequently struggle with scalability and do not effectively adapt to varying workload conditions. Researchers have responded by investigating meta-heuristic techniques, which have demonstrated improved performance through dynamic task scheduling optimization. Quantum-inspired algorithms have gained traction in task scheduling due to their ability to efficiently explore vast search spaces. Naik et al. [1] proposed a quantum-inspired particle swarm optimization (QPSO) approach for energy-efficient task offloading and resource allocation, reducing task completion time and energy consumption.

Similarly, Hasani Zade et al. [2] developed a fuzzy quantum atom search optimizer to address the task scheduling problem in cloud environments, resulting in a balanced trade-off between makespan and resource utilization. In a separate study, Priyanka et al. [3] utilized quantum-inspired differential evolution (QIDE) to enhance task offloading for energy and cost efficiency, surpassing conventional methods. Hussain et al. [4] devised a cost-aware quantum-inspired genetic algorithm for workflow scheduling in hybrid clouds, illustrating a balance between cost reduction and performance enhancement. Despite these advancements, numerous quantum-inspired models face challenges in accommodating dynamic workloads,

underscoring the need for hybrid solutions that incorporate learning mechanisms.

Overview of Energy-Efficient frameworks and load balancing approaches

Energy efficiency is crucial in cloud computing due to the rising environmental consequences and operational expenses associated with extensive data centers. Various frameworks have been proposed to enhance energy efficiency through effective virtual machine (VM) management, load balancing, and task scheduling.

Nature-inspired meta-heuristic algorithms, including genetic algorithms (GA), ant colony optimization (ACO), and particle swarm optimization (PSO), have been thoroughly investigated for load balancing and energy-efficient task scheduling. Li et al. [5] introduced a load-balancing framework employing meta-heuristic algorithms to reduce energy consumption while preserving task quality of service (QoS). Likewise, Veeramachaneni et al. [6] investigated quantum algorithms to enhance the integration of renewable energy in AI-driven data centres, resulting in improved resource utilization and sustainability. Deep learning models have been utilized to improve load-balancing efficiency. Chandrasiri and Meedeniya [9] developed an energy-efficient dynamic workflow scheduling model using deep learning, which resulted in enhanced task scheduling efficacy and reduced energy consumption. Furthermore, Kumaraswamy et al. [7] introduced a deep learning-based multi-objective framework that optimizes task scheduling and load balancing concurrently. Notwithstanding these initiatives, current methodologies often struggle to adapt dynamically to fluctuations in workload, resulting in inefficient resource management and increased operational expenses.

Summary of recent hybrid Meta-Heuristic and machine Learning-Based methods

Hybrid models that combine meta-heuristic algorithms with machine learning techniques have demonstrated significant potential in overcoming the constraints of independent methods. These models leverage the advantages of both paradigms to enhance task scheduling, energy optimization, and load balancing in cloud environments. Acharya et al. [12] introduced a hybrid golden search whale optimization algorithm that effectively balances makespan and energy consumption. Brahmam and Anand [11] developed a fused meta-heuristic model (VMMISD) that integrates security protocols and deep learning optimizations for efficient load balancing and task scheduling in cloud environments. Hussien et al. [25] developed a hybrid artificial gorilla troops optimizer combined with a honey-badger algorithm to address

intricate cloud scheduling issues, enhancing performance and resource utilization.

Additionally, Mikram et al. [29] introduced the HEPGA hybrid algorithm to tackle workflow scheduling challenges in cloud computing, exhibiting enhanced task scheduling efficacy. Notwithstanding these advancements, numerous hybrid methodologies experience heightened computational complexity and limited scalability in extensive environments, thereby necessitating the development of adaptive models that strike a balance between system performance and computational burden.

Recent advancements in Multi-Objective optimization and Energy-Aware scheduling

Recent multi-objective genetic algorithms for energy-aware task scheduling in cloud computing environments employ innovative and modern approaches to resource management. For instance, Li et al. Drastic workflow demands require scheduling tasks on fog computing systems, and tackling how to do so while assuring SLA is critical towards energy-efficient scheduling of tasks on fog computing systems [36]. Similarly, Li et al. proposed a hybrid bio-inspired algorithm for edge-cloud integrated computation to optimize task offloading and minimize energy consumption [37]. Additionally, Li et al. present a multi-objective task offloading method using the BBO algorithm in [38], which tackles the energy consumption and deadline constraints of task offloading in mobile edge computing, specifically the scheduling of tasks with stringent performance requirements. This indicates considerable advancement in the field of energy efficiency as it relates to multi-objective optimization.

Additional features include a multi-objective energy-aware task scheduling (MOEATS) solution for cloud environments by Nagalakshmi and Subramanian [39], which utilizes an algorithm based on orthogonal learning particle swarm optimization (PSO) to minimize both energy consumption and task execution time. Fang et al. A multi-objective evolutionary method for scheduling workflows over a multi-cloud system with energy, cost, and reliability targets [40]. The significance of these proposals lies in developing a hybrid algorithm that combines multiple optimization algorithms to manage multi-objective scheduling and energy-efficient resource management in dynamic cloud and edge computing environments.

Hybrid Bio-Inspired algorithms: A review in Energy-Efficient task offloading

In recent years, hybrid bio-inspired algorithms (HBAs) have received considerable attention in DECE for energy-efficient task offloading, particularly in edge-cloud and mobile edge computing (MEC) systems, where resources are highly distributed. Able to outperform some

previously-mentioned bio-inspired counterparts, these algorithms combine abilities found in a diverse set of bio-inspired techniques (for example Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO)) to optimize multiple objectives (resource utilization, energy consumption, task scheduling, etc.), which makes them very applicable for dynamic and heterogeneous computing environments. In this domain, significant work has been done by Li et al. in “A Hybrid Bio-inspired Algorithm for Edge-cloud Integrated Computation” [14]. It synergizes between edge and cloud resources to optimize energy while maintaining the optimal task execution time. Using a holistic combination of bio-inspired methods, it provides efficient offloading decisions, leading to gains in system responsiveness and resource utilization across a spectrum of computing platforms. We provide an insightful hybrid model that overcomes the trade-off between energy efficiency and performance demand in edge-cloud systems.

Similarly, Li et al. To address this critical issue, the BBO algorithm has also been utilized for multi-objective task offloading under deadline constraints in mobile edge computing [2]. Their focus on the instantaneous nature of deadline meetings makes task offloading even more significant in the context of real-time applications. Leveraging this BBO algorithm, the proposed system can explore vast solution spaces to strike a balance between energy consumption and task completion times, satisfying stringent deadlines while maximizing performance. This strategy is specifically helpful in mobile edge computing environments characterized by limited resources and time-critical tasks.

Limitations of existing approaches and research gaps

Despite the significant progress made in task scheduling, energy optimization, and load balancing, several limitations persist in existing approaches, highlighting research gaps that need to be addressed:

- *Inadequate Adaptability to Dynamic Workloads:* Many traditional and meta-heuristic approaches operate under static assumptions, failing to adapt to real-time workload variations [21]. Furthermore, this leads to inefficient resource allocation and a prolonged makespan, which is especially problematic when dealing with fluctuating task loads.
- *Suboptimal Energy Efficiency and VM Migration:* While meta-heuristic and machine learning approaches improve energy efficiency, excessive VM migrations and high computational overheads often degrade system performance. Intelligent virtual machine (VM) classification and workload prediction models are essential for minimizing unnecessary migrations and optimizing energy efficiency [22–25].

- *Limited Integration of Reinforcement Learning for Real-Time Adaptability*: Although hybrid models have demonstrated improved efficiency by integrating meta-heuristics with machine learning, most frameworks fail to incorporate deep reinforcement learning (DRL), which can dynamically adapt to workload variations and optimize task scheduling in real-time [26–31].
- *High Computational Complexity and Scalability Issues*: The high computational complexity of many existing hybrid frameworks restricts their use in expansive cloud environments. Processing high-dimensional workloads and dynamic task arrivals introduces complexity, making scalability a persistent issue [5–11].
- *Lack of Comprehensive Multi-Objective Optimization*: Many task scheduling and energy optimization models focus on single-objective optimization, often neglecting conflicting objectives such as minimizing makespan, reducing energy consumption, and maximizing resource utilization. A robust multi-objective framework is required to balance these objectives effectively [15–21].

Comparative analysis of existing models and proposed framework

A comparative analysis has been conducted to highlight the advantages of the proposed Quantum-Inspired Hybrid Reinforcement Learning and Multi-Objective Optimization Framework (QHRMOF) in comparison to existing methodologies. Table 1 below summarizes essential features, including adaptability to fluctuating

workloads, energy efficiency, real-time learning, and multi-objective optimization.

Table 1 illustrates that the proposed QHRMOF framework overcomes the deficiencies of current models by integrating QIEA with DRL to attain dynamic workload adaptability, real-time learning, and multi-objective optimization. The framework efficiently reduces makespan, optimizes energy usage, and improves load balancing, rendering it a robust solution for extensive cloud environments.

Proposed framework: QHRMOF

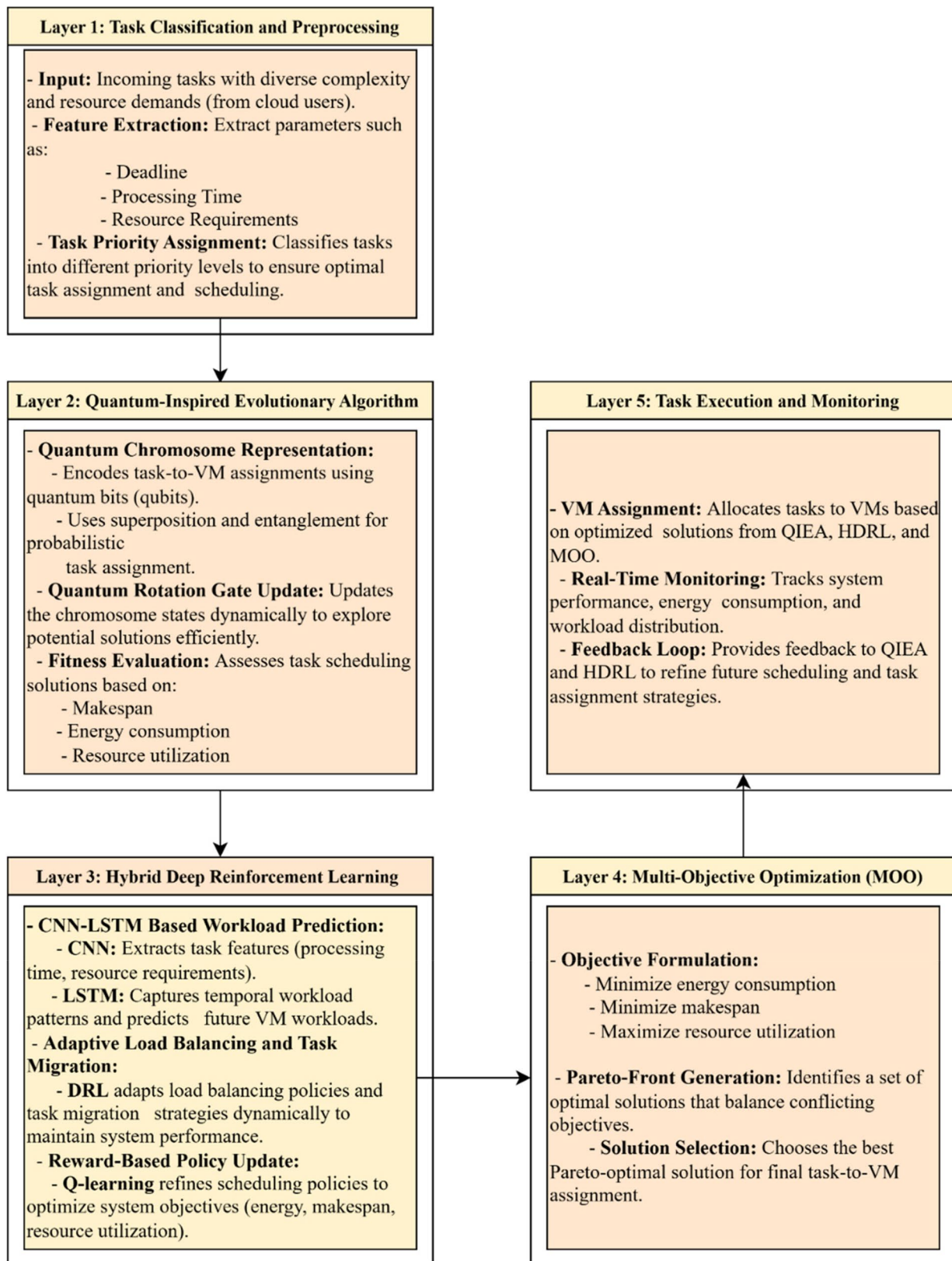
The proposed Framework QHRMOF aims to enhance task scheduling and resource allocation in cloud environments by integrating quantum-inspired algorithms, deep reinforcement learning, and multi-objective optimization methods. The proposed framework effectively addresses energy efficiency challenges, workload forecasting, and dynamic task migration while optimizing multiple conflicting objectives.

Architecture of QHRMOF

The proposed Framework QHRMOF amalgamates quantum-inspired evolutionary algorithms (QIEA), deep reinforcement learning (DRL), and multi-objective optimization (MOO) to optimize task scheduling and resource management in cloud environments. The architecture ensures efficient workload allocation, reduced energy usage, and shortened makespan while maintaining high resource utilization [1–7]. Figure 1 presents the architecture of the proposed Framework QHRMOF.

Table 1 Comparative analysis of existing models and proposed framework

Reference	Dynamic Workload Adaptability	Energy Efficiency	Real-Time Learning	Multi-Objective Optimization	Low Computational Overhead
Naik et al. [1]	No	Yes	No	Partial	Yes
Hasani Zade et al. [2]	No	Yes	No	Partial	Yes
Priyanka et al. [3]	No	Yes	No	No	Yes
Hussain et al. [4]	No	Yes	No	Partial	Yes
Li et al. [5]	No	Yes	No	No	Yes
Veeramachaneni et al. [6]	Partial	Yes	No	Partial	Yes
Komarasamy et al. [7]	Partial	Yes	No	Yes	No
Brahmam and Anand [11]	Partial	Yes	Partial	Yes	No
Acharya et al. [12]	No	Yes	No	Partial	Yes
Hussien et al. [25]	Partial	Yes	No	Partial	No
Mikram et al. [29]	Partial	Yes	No	Partial	No
Li et al. [36]	No	Yes	No	Yes	No
Li et al. [37]	No	Yes	No	Yes	No
Li et al. [38]	No	Yes	No	Yes	No
Proposed QHRMOF	Yes	Yes	Yes	Yes	Yes

**Fig. 1** The architecture of the proposed Framework QHRMOF

Algorithm for proposed QHRMOF framework

```

START
// Step 1: Task Classification and Preprocessing
Input: Incoming tasks from cloud users (task list)
  For each task in tasklist:
    Extract task features (deadline, processing time, resource requirements)
    Classify tasks based on priority (High, Medium, Low)
    Store classified tasks into corresponding priority queues
// Step 2: Quantum-Inspired Evolutionary Algorithm (QIEA)
Initialize Quantum Chromosomes (Qchromosomes)
  For each generation in maxgenerations:
    // Quantum Rotation Gate Update
    For each quantum chromosome in Qchromosomes:
      Apply quantum gates (superposition, entanglement) to explore task-to-VM assignments
    // Fitness Evaluation
    For each quantum chromosome in Qchromosomes:
      Calculate fitness based on:
        - Makespan (task completion time)
        - Energy consumption (power usage)
        - Resource utilization (CPU, memory usage)
      // Select best solutions (task-to-VM assignments) based on fitness
// Step 3: Hybrid Deep Reinforcement Learning
Initialize CNN and LSTM models for workload prediction
  For each task in tasklist:
    // CNN-based Feature Extraction
    Extract task features using CNN (processing time, resource requirements)
    // LSTM-based Temporal Workload Prediction
    Predict future VM workloads using LSTM
  // Load Balancing and Task Migration
  For each VM in VMpool:
    If VM is overloaded OR underloaded:
      // Apply DRL-based task migration
      Calculate optimal task migration strategy (based on predicted workload and task priority)
      Migrate tasks to other VMs based on DRL policy
// Step 4: Multi-Objective Optimization
Initialize Multi-Objective Optimization
  // Define optimization objectives:
  Minimize energy consumption
  Minimize makespan
  Maximize resource utilization
  // Generate Pareto Front
  Generate Pareto Front from available solutions:
    - Calculate energy, makespan, and resource utilization for each solution
    - Identify an optimal set of trade-off solutions
  // Select Best Pareto-Optimal Solution
  Select the best solution from the Pareto-optimal set (task-to-VM assignment)
// Step 5: Task Execution and Monitoring
Allocate tasks to VMs based on the best Pareto-optimal solution
  // Real-Time Monitoring
  For each VM in VMpool:
    Monitor energy consumption
    Monitor task progress and completion time
    Track VM performance metrics (CPU, memory usage)
  // Feedback Loop
  Collect feedback (task performance, energy consumption, VM load)
  Update QIEA and HDRL models with feedback to refine future task assignments and scheduling strategies.
END

```

Algorithm 1. Presents the steps for the Proposed Framework (QHRMOF)

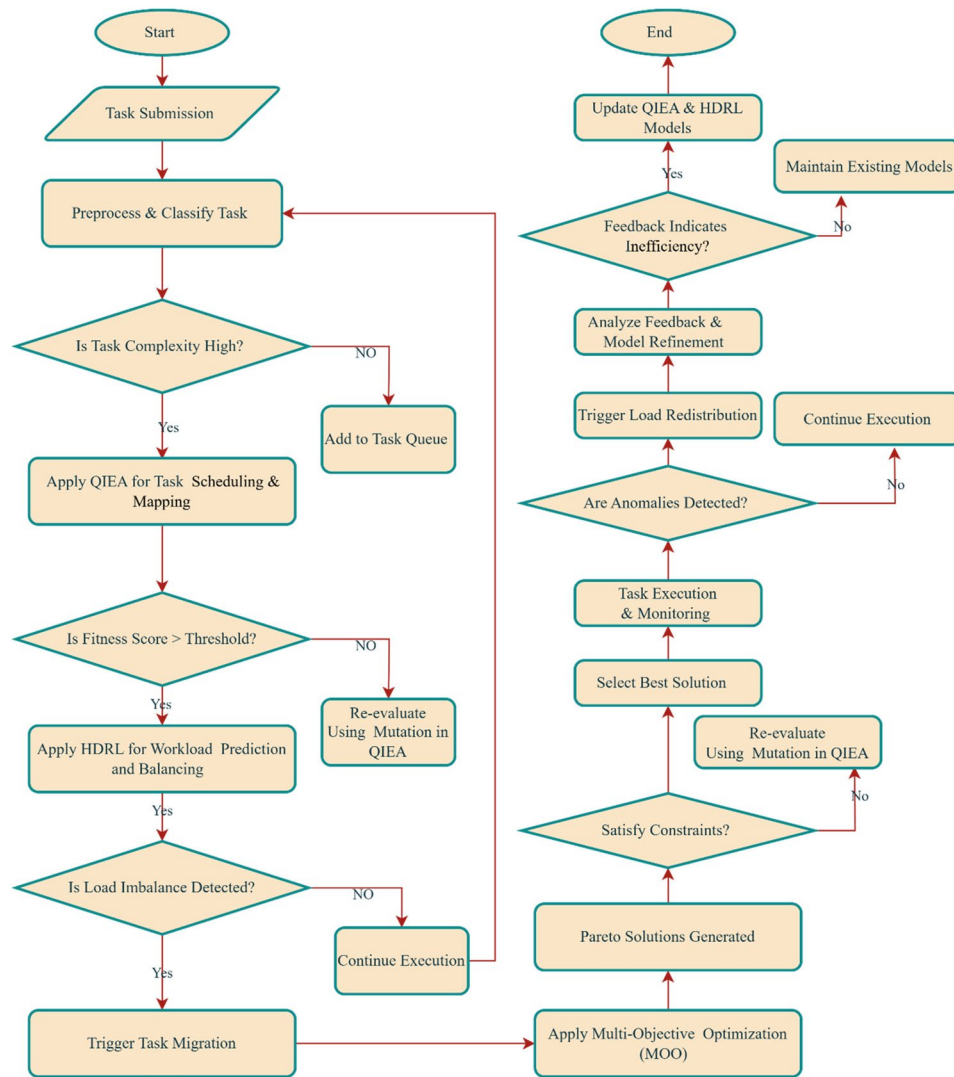


Fig. 2 Flow chart of proposed QHRMOF

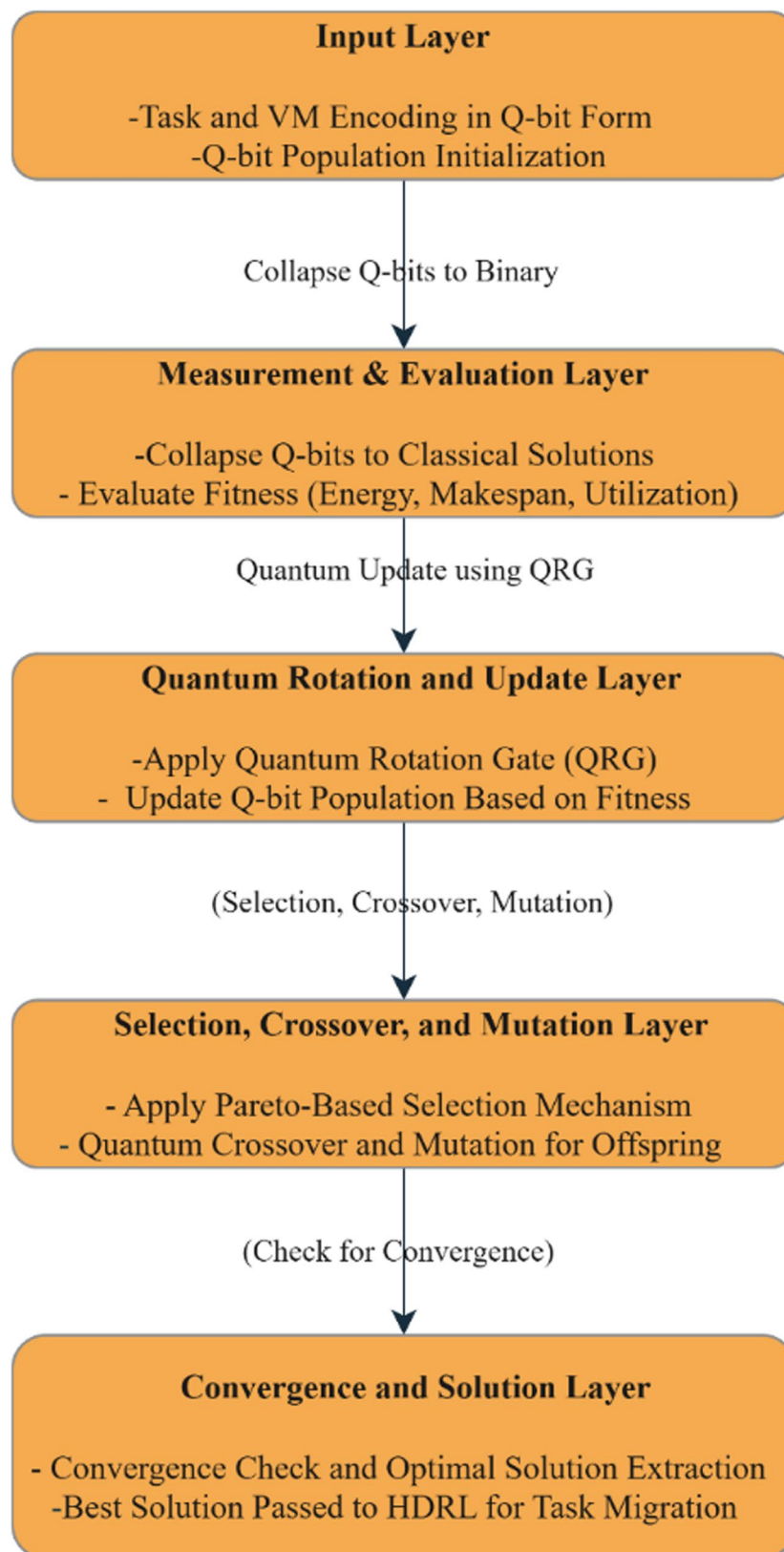
Key components and workflow of proposed QHRMOF

The proposed QHRMOF utilizes a layered architecture that separates task scheduling and resource allocation into multiple functional layers [2–11], as illustrated in Fig. 2. The comprehensive workflow of QHRMOF includes the following.

- *Task Classification and Preprocessing:* This layer manages incoming cloud tasks by extracting pertinent parameters, including deadlines, processing times, and resource requirements, and assigning priorities to ensure optimal task scheduling [12].
- *QIEA:* This module utilizes quantum principles, including superposition and entanglement, to represent task-to-VM assignments in quantum chromosomes, enhancing solution search efficiency. The module assesses the efficacy of the solutions

based on makespan, energy consumption, and resource utilization [13]–[14].

- *HDRL:* The HDRL module predicts future VM workloads using a combination of CNN (for feature extraction) and LSTM (for temporal prediction). It dynamically adapts the load-balancing and task-migration strategies to optimize system performance. The Q-learning algorithm refines policies to enhance task scheduling decisions. This module utilizes quantum principles, including superposition and entanglement, to represent task-to-VM assignments in quantum chromosomes, enhancing solution search efficiency. The module assesses the efficacy of the solutions based on makespan, energy consumption, and resource utilization [15].
- *MOO:* This layer addresses multiple conflicting objectives, such as energy consumption, makespan, and resource utilization. It uses Pareto-front

**Fig. 3** A High-level Architecture of QEA in the Proposed QHRMOF

generation to identify optimal solutions and select the best one for the final task-to-VM assignment.

- *Task Execution and Monitoring:* When tasks are assigned to virtual machines, this layer oversees real-time performance and monitors key metrics, including energy consumption, load distribution, and task progress. The system additionally transmits performance data to QIEA and HDRL for subsequent optimization in future scheduling [16–19].

Quantum-Inspired evolutionary algorithm (QIEA)

The QHRMOF relies on the QIEA to optimize resource allocation and task scheduling in a multi-cloud setting. By utilizing the concepts of quantum superposition and entanglement, QIEA improves exploration and exploitation capabilities to attain optimal scheduling while balancing energy consumption, makespan, and resource utilization [20–23].

Role of QIEA in proposed QHRMOF

The primary role and responsibility of QIEA in the proposed QHRMOF are as follows.

- *Efficiently explore the search space:* Encoding solutions to the task scheduling problem in Q-bit form enables the simultaneous exploration of multiple solutions [24].
- *Optimize task-resource mapping:* This ensures that tasks are dynamically assigned to virtual machines

(VMs) with the least possible delay and energy overhead.

- *Prevent premature convergence:* Applying quantum principles helps preserve the population's diversity, preventing the algorithm from becoming mired in a state of locally optimized behaviors [25].

High-Level architecture of QIEA in QHRMOF

Figure 3 presents a high-level architecture of QIEA in the proposed QHRMOF. The detailed workings of these layers are described below [24–27].

- *Layer 1: Input and Initialization Layer:* This layer is responsible for Tasks and VM Encoding. In layer 1, tasks and available virtual machines (VMs) are represented using quantum bits (Qubits) to leverage quantum principles such as superposition. Each qubit can represent multiple potential states concurrently, denoted by Eq. 1.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

Where α and β are complex probability amplitudes.

Equation 1 facilitates the concurrent examination of multiple task-to-VM mappings, offering a robust method for investigating an expansive solution space. A population of Q-bit chromosomes is randomly initialized, creating an initial pool of candidate solutions. The population size, represented as P , comprises several quantum chromosomes, with each chromosome symbolizing a potential task-to-VM mapping. The initial population serves

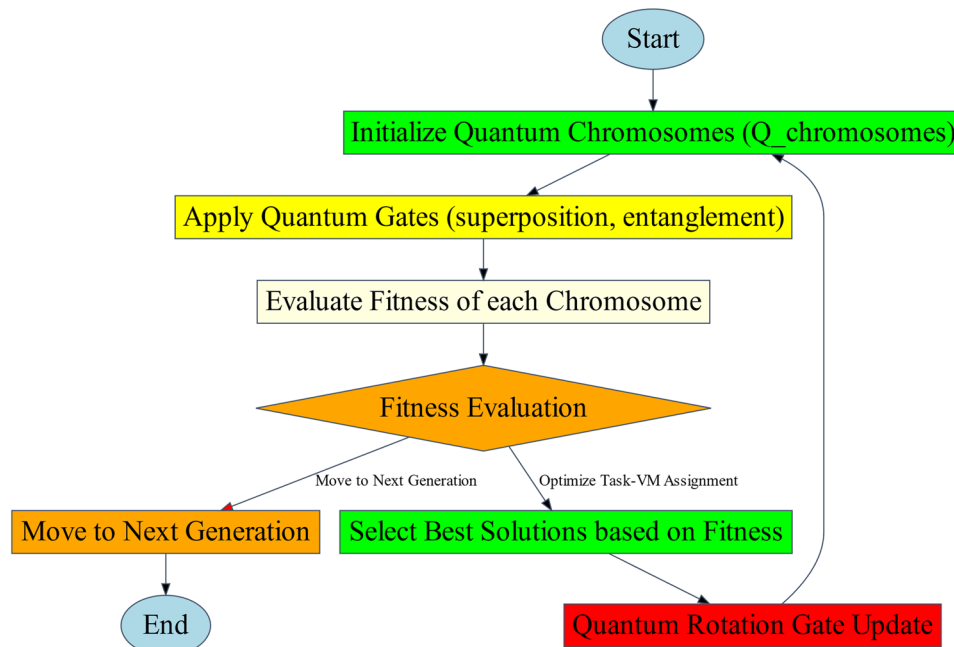


Fig. 4 Flow chart of QIEA

as the foundation for the optimization process, during which quantum operations will refine and enhance the solution through multiple iterations. This method significantly improves the system's exploration capabilities, thereby increasing the likelihood of identifying optimal or near-optimal solutions.

- *Layer 2: Measurement and Fitness Evaluation Layer:*

In Layer 2, the quantum solutions stored in Q-bits are measured or “collapsed” into standard binary values. In this process, the value of each Q-bit is used to determine its state, which then converts its superposition into one of two possible outcomes: 0 or 1. The likelihood of transitioning into each state is determined by the amplitudes α and β of the quantum state, with the probabilities of achieving 0 or 1 expressed by Eq. 2.

$$P(0) = |\alpha|^2 \text{ and } P(1) = |\beta|^2 \quad (2)$$

$$F(X) = w_1 f_1(X) + w_2 f_2(X) + w_3 f_3(X) \quad (3)$$

After the collapse of Q-bits into classical solutions, these solutions are assessed according to a multi-objective fitness function. The fitness assessment encompasses three fundamental objectives: energy consumption $E(X)$, makespan $T(X)$, and resource utilization $U(X)$. Each of these objectives is quantified for a specific solution X . The comprehensive fitness function, $F(X)$, is expressed as a weighted sum of the individual goals, as denoted by Eq. 3. Here, w_1, w_2 , and w_3 : Weight parameters.

The objective of layer 2 is to reduce energy consumption and makespan while enhancing resource utilization, achieved via the multi-objective fitness function. This layer provides a framework for assessing and prioritizing various task-to-VM allocations, directing the optimization process toward superior solutions based on established performance criteria. Figure 4 presents the flowchart of QIEA.

- *Layer 3: Quantum Rotation and Update Layer:* In Layer 3, the quantum population (the collection of Q-bits denoting potential task-to-VM mappings) is revised via a quantum rotation process, which augments the exploration of the solution space. The quantum rotation gate (QRG) is utilized on each qubit in the population to alter its state, enabling the algorithm to navigate the solution space more effectively [21–17]. The quantum rotation gate functions by modifying the state of each Q-bit by applying a rotation matrix. The standard representation of this update for each Q-bit i is expressed through the matrix operation: $(\alpha_i' \beta_i')$

denotes the revised values of these amplitudes after the rotation (Eq. 4).

$$\begin{pmatrix} \alpha_i' \\ \beta_i' \end{pmatrix} = \begin{pmatrix} \cos(\Delta\theta) & -\sin(\Delta\theta) \\ \sin(\Delta\theta) & \cos(\Delta\theta) \end{pmatrix} \begin{pmatrix} \alpha_i \\ \beta_i \end{pmatrix} \quad (4)$$

$$\Delta\theta = \eta \times \text{sgn}(f_{\text{best}} - f_i) \times \theta_{\text{max}} \quad (5)$$

A key function in quantum computing is the rotation matrix, which is designed to modify each qubit's quantum state, allowing researchers to experiment with different solutions by adjusting the probabilities of each state. The rotation angle ($\Delta\theta$) dictates the extent of the modification and can be calculated using Eq. 5.

The learning rate (η) determines the extent of rotation. $\text{sgn}(f_{\text{best}} - f_i)$ denotes the sign function, which signifies the direction of change based on the disparity between the optimal solution identified thus far (f_{best}) and the fitness of the current solution. θ_{max} represents the maximum rotation angle, which constrains the extent to which the state of each Q-bit can alter in a single update. This quantum rotation enables the algorithm to optimize the solution by modifying the quantum state of each candidate solution according to its fitness [24–29]. This rotation allows the algorithm to dynamically adjust the search process, thereby increasing the likelihood of converging on an optimal or near-optimal solution while circumventing premature convergence.

- *Layer 4: Selection, Crossover, and Mutation Layer:* This layer is an essential element of the quantum-inspired optimization algorithm. The process involves identifying optimal solutions, generating offspring solutions through crossover, and applying mutation to maintain diversity and prevent premature convergence of the algorithm [28–31]. The details of each component are as follows.
- *Selection (Pareto-Based Selection Mechanism):* Selection is a process that identifies the optimal solutions (chromosomes) to contribute to the subsequent generation. A Pareto-based selection mechanism is employed in this instance. Solutions are chosen according to Pareto dominance, whereby a solution is deemed superior if it is not surpassed by any other solution in terms of the objectives. In multi-objective optimization problems, this facilitates the identification of solutions that achieve an optimal equilibrium among various objectives (e.g., minimizing energy consumption and maximizing resource utilization). Pareto dominance ensures the simultaneous optimization of multiple objectives, enabling the selection of optimal solutions, known as the Pareto front, for subsequent processing [32].

- **Quantum Crossover:** A Crossover is a genetic operator that creates offspring solutions by combining two parent solutions (chromosomes). The quantum crossover mechanism employs a crossover operation inspired by quantum mechanics and performed between two high-fitness parental solutions to produce offspring chromosomes. The following equations, 6 and 7, correspond to the quantum crossover phenomenon:

$$\alpha'_i = \alpha_i * \alpha_j - \beta_i * \beta_j \quad (6)$$

$$\beta'_i = \alpha_i * \beta_j + \beta_i * \alpha_j \quad (7)$$

Where: α_i, β_i They are the quantum states (amplitudes) of the parent chromosome. α_j and β_j are the quantum states of the j^{th} parent chromosome. α'_i and β'_i are the quantum states of the offspring chromosome.

The offspring's quantum states are generated by combining the quantum states of the parent chromosomes (α and β). This creates new potential solutions (offspring) with characteristics of both parents, potentially leading to better solutions by combining their strengths.

- **Mutation:** The mutation is another significant genetic operator responsible for introducing randomness into the population. It modifies the offspring's chromosomes to preserve diversity and prevent the algorithm from prematurely converging to a local optimum. In optimization problems, this is of the utmost importance because premature convergence can result in less-than-optimal solutions [30–34].

This layer is where the mutation is applied with a specific probability. P_m , which determines the frequency with which the mutation occurs over time. The mutation can flip the quantum bits, which means it can change the state of α or β in the chromosomes. This will increase the population's diversity and also assist in exploring the solution space more efficiently [35].

- **Layer 5: Convergence and Solution Extraction**

Layer: This layer is essential for assessing when the algorithm has effectively converged to a solution and identifying the optimal solution for subsequent decision-making processes [36–38]. It comprises two fundamental elements: Convergence Check and Optimal Solution Extraction.

- **Convergence Check:** The convergence check ensures that the optimization process has achieved an optimal or near-optimal solution, indicating that the algorithm terminates when further progress becomes negligible.
- **Termination Condition:** The algorithm terminates when the variation in fitness values between

consecutive iterations falls below a specified threshold ϵ (epsilon), indicating that the population has converged and that further iterations will not yield substantial enhancements. Equation 8 presents the formula for checking convergence.

$$|F(X_t) - F(X_{t-1})| < \epsilon \quad (8)$$

Where: $F(X_t)$ represents the fitness value of the solution at iteration t . The fitness value of the solution at iteration $t-1$ is represented by $F(X_{t-1})$. Here, ϵ is a minor, set threshold value.

The optimization process is deemed converged when the disparity in fitness values between successive iterations falls below ϵ , at which point the algorithm may terminate. This conserves computational resources and mitigates overfitting or superfluous calculations.

- **Optimal Solution Extraction:** The algorithm retrieves the best possible solutions after reaching a state of convergence [39]. In the context of a multi-objective optimization problem, the best solutions are located on the Pareto front [40]. This means they offer the best trade-offs between competing objectives (for example, energy consumption, resource utilization, and makespan).
- **Pareto-Optimal Solutions:** These solutions can't be improved without worsening another goal. They are determined to be the last contenders after the algorithm has converged.
- **Passing Solutions to HDRL:** The optimal Pareto solutions are transmitted to the HDRL module. The HDRL module manages dynamic load balancing and task migration. It employs Pareto-optimal solutions to enhance task allocation to VMs and oversee real-time load balancing.

This step ensures that the final optimized solutions (evaluated through the quantum-inspired optimization process) are ready to be utilized by the system's task scheduling and load-balancing mechanisms.

Hybrid deep reinforcement learning

The HDRL module integrates CNNs and LSTM networks to forecast workload patterns and categorize VMs. Subsequently, an RL agent uses the identified patterns to dynamically equilibrate system load and migrate tasks in real-time, thereby improving the efficacy of cloud-based resource management. Figure 5 presents the High-Level Architecture of HDRL in the Proposed QHRMOF.

CNN and LSTM for workload prediction and VM classification

The working of CNN-LSTM in the Proposed framework is presented in Fig. 6, with the following details.

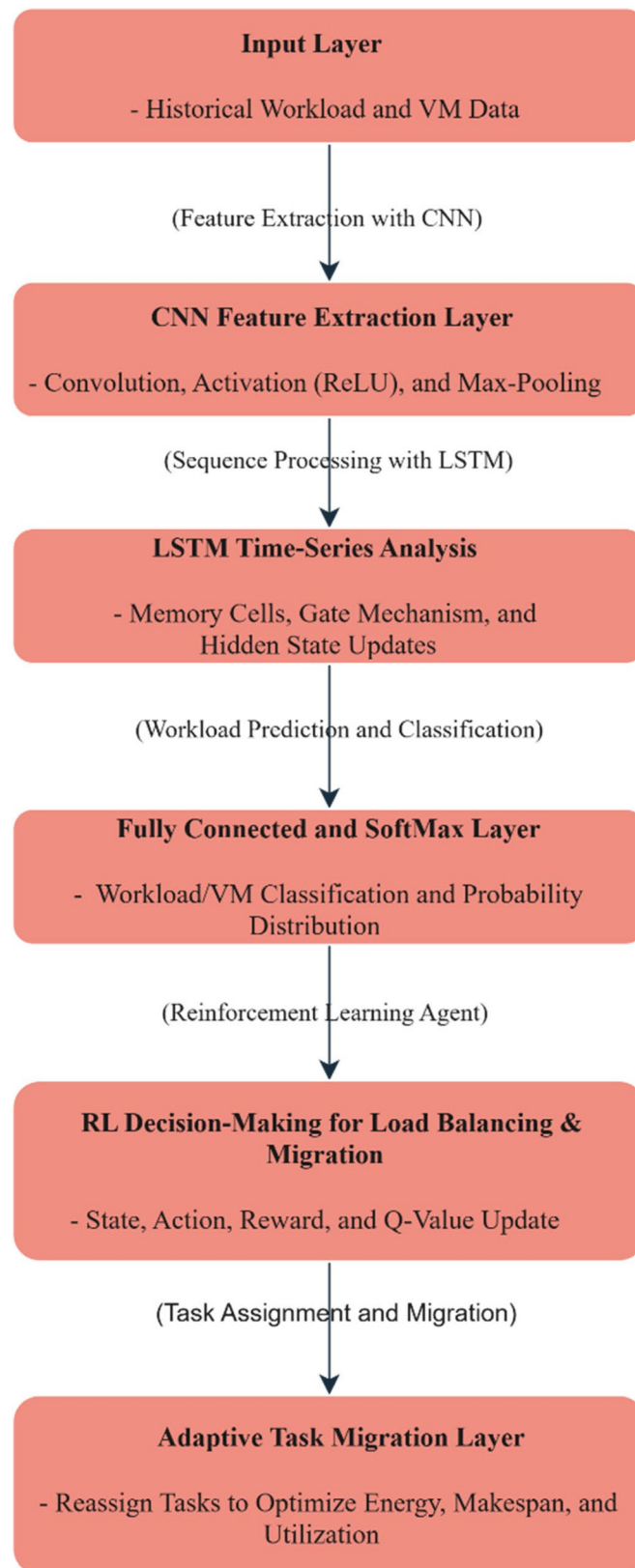
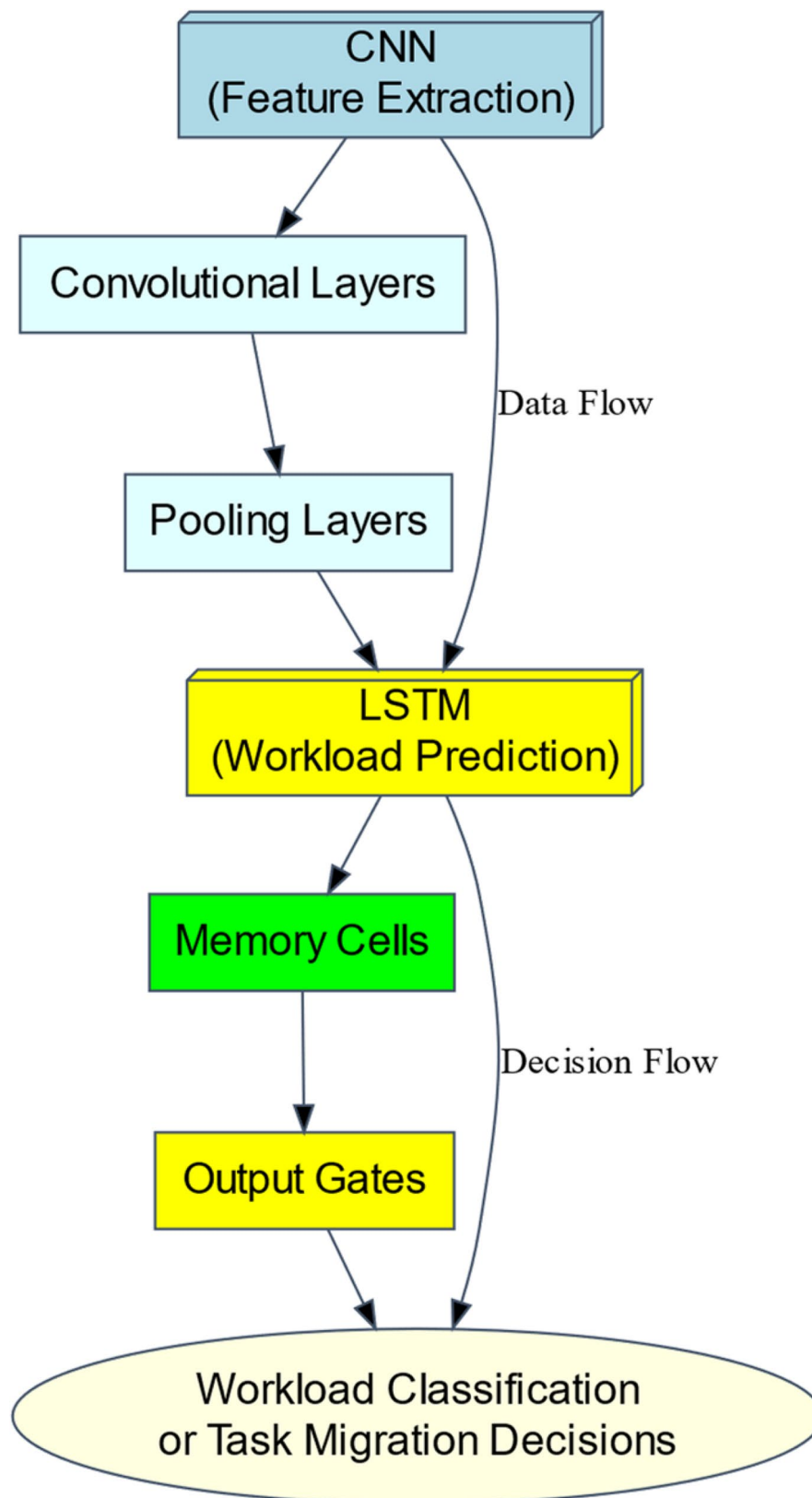


Fig. 5 A High-level Architecture of the Hybrid Deep Reinforcement Learning Architecture in the Proposed QHRMOF

**Fig. 6** Flowchart CNN and LSTM for Workload Prediction and VM Classification

I. Input Layer:

- **Historical Workload Data:** A matrix of workload patterns (such as CPU, RAM, and disk utilization over time).
- **VM Performance Logs:** Time series data that captures resource consumption metrics from virtual machines. Equation 9 is a representation of the input data, where x_i Represents the resource metrics of the VM at time t.

$$X_t = [x_1, x_2 \dots \dots \dots, x_n] \quad (9)$$

II. Layer 1: CNN Feature Extraction

- **Convolutional Layer:** The CNN derives spatial characteristics from the workload time-series data.
- **Feature Map Generation:** Convolutional filters are applied to the data to generate feature maps, as shown in Eq. 10.

$$F_{ij}(k) = \sigma \left(\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} w_{mn}(k) \cdot x(i+m)(j+n) + b(k) \right) \quad (10)$$

Where: $F_{ij}(k)$ represents feature map at position (i, j), $w_{mn}(k)$ represents kernel weight, $b(k)$ represents the bias term.

III. Layer 2: Max-Pooling Layer

- **Down-sampling:** Convolutional filters are applied to the data to generate feature maps, as shown in Eq. 11. Where $P_{ij}(k)$ Represents the probability for (i, j).

$$P_{ij}(k) = \max_{m,n} F(i+m)(j+n)(k) \quad (11)$$

IV. Layer 3: LSTM for Time Series Analysis

- **LSTM Process:** LSTM analyses time-series data, preserving essential information through memory cells as presented by Eq. 12.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (12)$$

- **Forget, Input, and Output Gates:** The Forget, Input, and Output Gates are responsible for managing the cell's state and controlling the flow of information, as presented in Eqs. 13 and 14. Where C_t represents cell state, h_t hidden state, and O_t Output state.

$$C_t = [(f_t \cdot C_{t-1}) + (i_t \cdot \dot{C}_t)] \quad (13)$$

$$h_t = [O_t \cdot \tanh(C_t)] \quad (14)$$

V. Layer 4: Fully Connected (Dense) Layer.

- **Workload Prediction and VM Classification:** To generate probabilities, the fully connected layer maps the extracted features to workload predictions or virtual machine classifications using the SoftMax method, as presented in Eq. 15. Where W_d and b_d represents the weights and biases of the dense layer.

$$y = \text{SoftMax}(W_d \cdot h_t + b_d) \quad (15)$$

VI. Output Layer

- Based on the learned features, the model predicts the workload category or classifies virtual machines.

Adaptive Decision-Making for dynamic load balancing and task migration

Adaptive Decision-Making is the fundamental mechanism in the HDRL module. It utilizes reinforcement learning (RL) to ensure effective load balancing and task migration in real time based on expected workload patterns. Deep learning integrated with a Markov Decision Process (MDP) drives this process for workload prediction using Q-learning.

- **Reinforcement Learning for Decision-Making.**

Reinforcement learning is crucial to the HDRL module's decision-making process for resource management, encompassing tasks such as task migration and load balancing. The RL agent gradually learns through interaction with the environment, receiving feedback (in the form of rewards) derived from its actions. This enables the system to make informed decisions that enhance productivity and efficiency.

- **Markov Decision Process (MDP) Formulation.**

An MDP, a mathematical model for sequential decision-making problems where the outcome depends on the current state and the action taken, is used to frame the decision-making process. MDPs consist of four primary parts:

- State Space (S):** This represents the system's current state, encompassing both the workload state and VM configurations. The workload state may encompass resource consumption metrics (e.g., CPU usage, RAM), while VM configurations denote the current operating conditions and allocation of virtual machines (VMs).
- Action Space (A):** The action space represents the diverse potential actions the agent may undertake to alter the system's state. Within the framework of load

balancing and task migration, the subsequent actions may be exemplified:

- *Task migration*: Task migration transfers tasks from one virtual machine to another.
 - *VM Reassignment*: In VM reassignment, resources are reallocated, or tasks are reassigned to different virtual machines (VMs).
 - *Scaling*: Scaling refers to modifying the number of VMs or the resources they use.
- c) *Reward Function (R)*: The reward function provides feedback on the effectiveness of the agent's actions. The system's performance is evaluated after an action is taken (such as task migration or scaling). The reward is based on how well the system's efficiency, load balancing, and overall performance are improved. Higher rewards are given when the system operates more efficiently (e.g., fewer overloaded virtual machines and enhanced resource utilization).
- d) *Transition Function*: The function delineates the probability of transitioning from one state (S) to another state (S') contingent upon a specific action (A) (Eq. 16).

$$T(O(f)) = (P(S'|S, A)) \quad (16)$$

For instance, when a task is transferred from an overloaded virtual machine to an underloaded virtual machine, the transition function assesses the probability of achieving improved load distribution and reduced resource utilization.

- *Q-Learning with Neural Network*.

Q-learning is a type of reinforcement learning used to determine the optimal action to take in a given state. The Q-value denotes the anticipated rewards for executing a particular action in a specified state. Gradually, the agent optimizes its actions by adjusting its Q-values in response to the rewards it receives. The Q-value update rule is presented by Eq. 17.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (17)$$

Where: $Q(s_t, a_t)$ Represents the Q-value for the current state and action (s_t, a_t) , α represents the learning rate, r_t represents rewards towards time t , γ Represents the discount factor, which assesses the significance of upcoming rewards.

- *Policy and Action Selection*.

The RL agent's method for selecting actions based on the current state is referred to as the policy. Exploration and exploitation are balanced using the Epsilon-Greedy strategy:

- a) *Exploration*: The agent tries random actions with a probability of ϵ to discover potentially better actions. Based on current knowledge, the agent chooses the most well-known action (the one with the highest Q-value) with a probability of $(1-\epsilon)$.
 - b) This equilibrium guarantees that the agent stays away from less-than-ideal solutions and keeps looking for better ones.
- *Task Migration Strategy*.

By moving tasks between virtual machines (VMs) according to anticipated workload patterns, the task migration strategy aims to maximize resource utilization. This is how it operates:

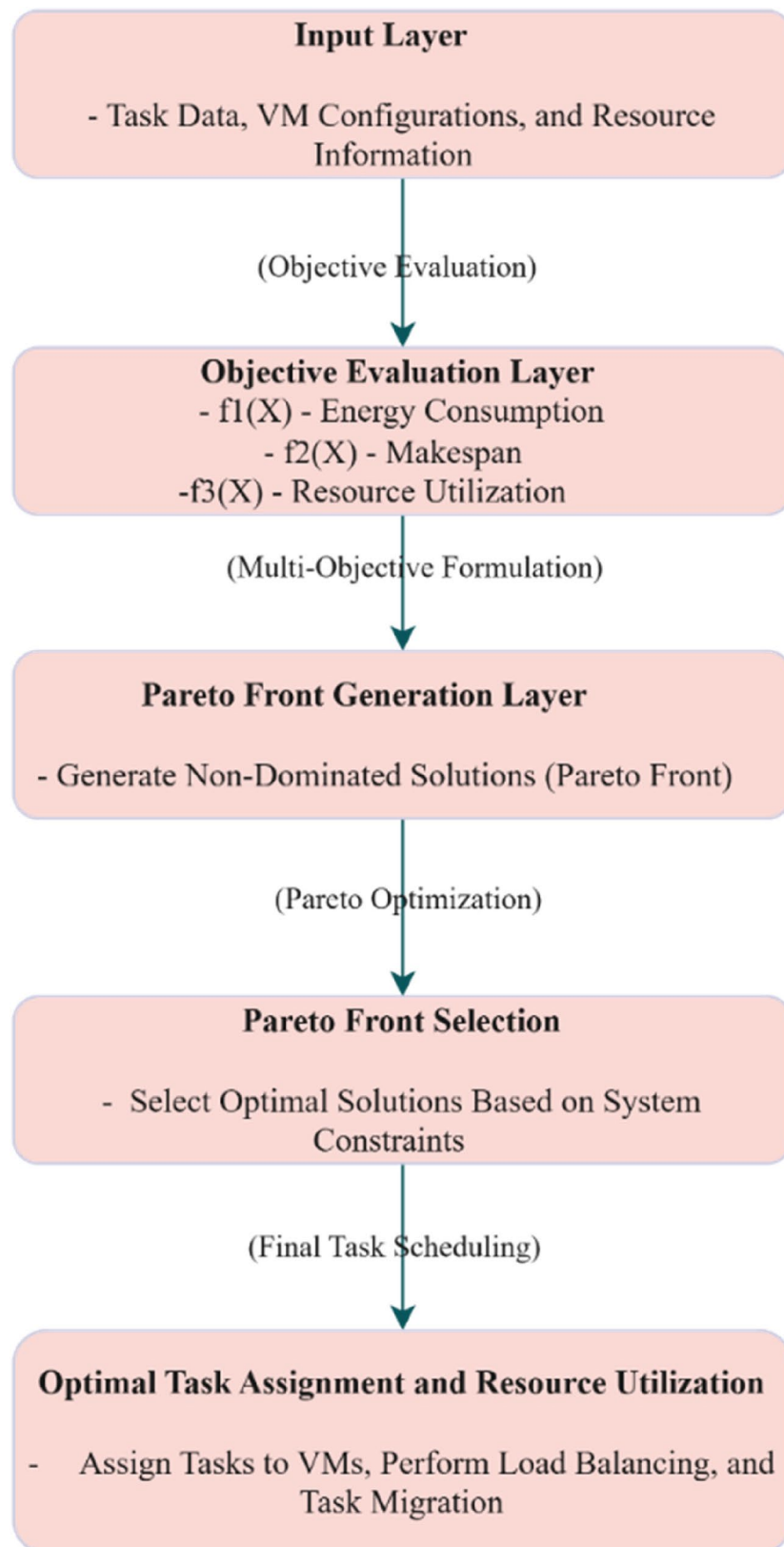
- a) *Predicted Workload*: The system predicts which VMs are likely to become overloaded and which are underutilized. This prediction is based on data from CNNs and LSTMs, which are used to forecast workload patterns.
- b) *Task Migration*: When a VM is predicted to become overloaded (i.e., its resource usage is too high), tasks are migrated to underutilized VMs. This migration helps distribute the workload evenly throughout the system.
- c) *Objective*: The goals of the task migration strategy are as follows.
 - *Minimize makespan* (the time required to complete all tasks).
 - *Reduce energy consumption* (by efficiently distributing tasks).
 - *Maximize resource utilization* by ensuring that VMs are fully utilized without overloading any single VM.

Multi-Objective optimization

The MOO module within the Proposed QHRMOF is essential for reconciling the competing objectives present in cloud resource management. These objectives often necessitate trade-offs, and the MOO module ensures that the system effectively manages these trade-offs while maintaining superior overall performance. Figure 7 presents a layered architecture for MOO in QHRMOF.

Figure 8 presents the Flowchart. The primary objectives of the MOO module within the QHRMOF framework are as follows.

- *Energy Consumption Minimization*: The goal is to reduce power consumption across all Virtual Machines (VMs). Cloud computing environments

**Fig. 7** Layered Architecture for MOO in QHRMOF

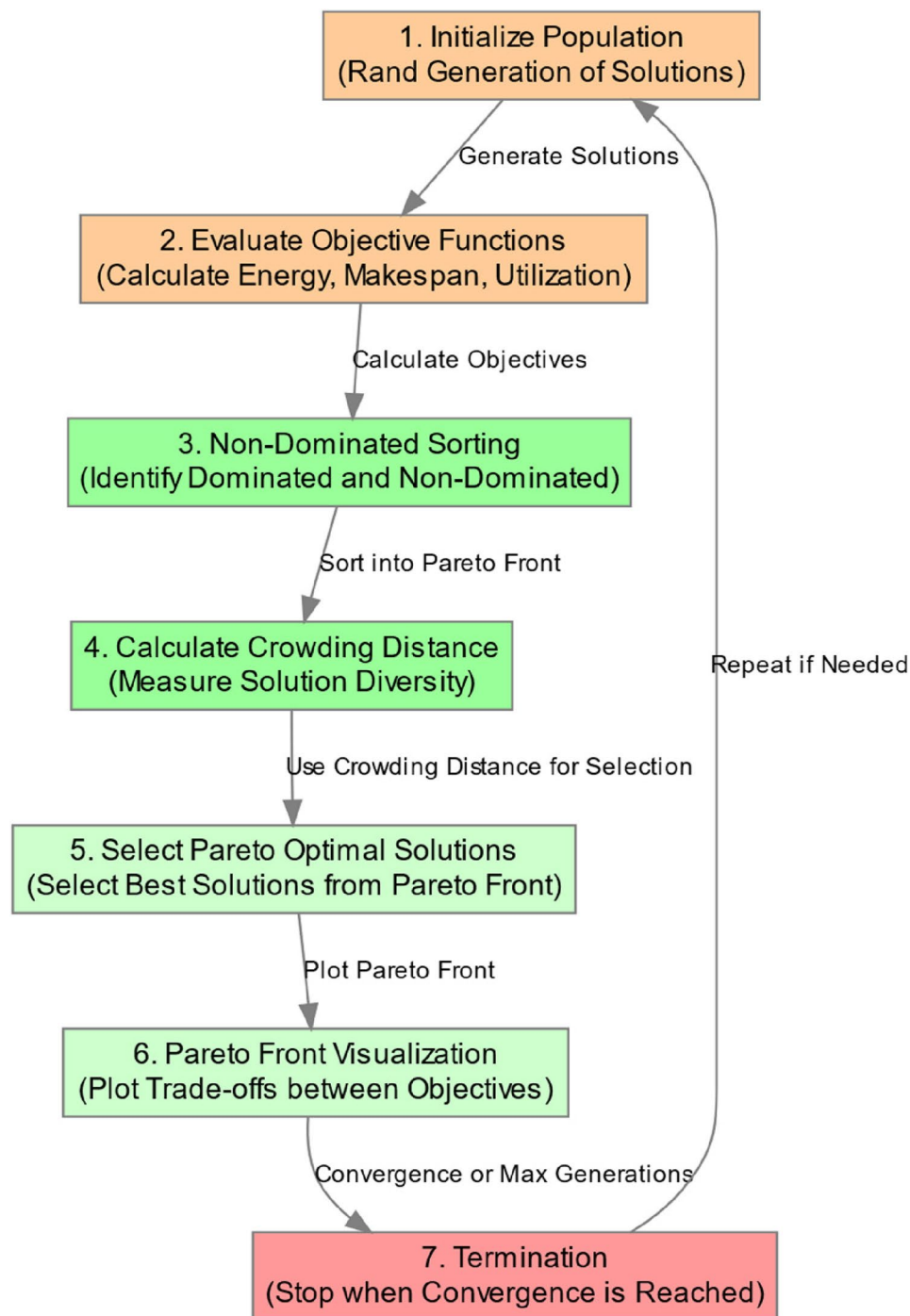


Fig. 8 Flowchart of the process of Multi-Objective Optimization (MOO) and the generation of the Pareto front for balancing conflicting objectives

typically consume a significant amount of energy, and optimizing energy consumption reduces operational costs and the system's environmental footprint. This is accomplished by lowering idle VM power consumption and optimizing task placement to prevent the overuse of specific VMs, thereby spreading energy consumption throughout the system.

- *Makespan (Minimization)*: Makespan denotes the total execution time required to complete all system tasks. The MOO module reduces time spent processing tasks by effectively balancing workloads across VMs and minimizing bottlenecks. The system enhances responsiveness and efficiency by reducing the makespan, leading to faster task completion and improved overall service delivery.

- **Resource Utilization (Maximization):** The aim is to optimize the utilization of computational resources available in the cloud. Optimally utilizing available resources, including CPU, memory, and storage, ensures the system operates at its maximum capacity, preventing underutilization and excessive load on virtual machines (VMs). Efficient resource allocation is crucial for maintaining equilibrium in the cloud environment, minimizing waste, and ensuring resource availability for new tasks as needed.

Optimization of conflicting objectives

- **Problem Formulation:** In a cloud environment with multiple objectives, task scheduling and virtual machine allocation must be done to minimize conflicting objectives. The formulation of the optimization problem is presented in Eq. 18.

$$\forall i \in \{1, 2, \dots, k\}, f_i(X_1) \leq f_i(X_2) \text{ and } \exists j \in \{1, 2, \dots, k\} \text{ such that } f_j(X_1) < f_j(X_2) \quad (22)$$

$$\text{Min /Max } F(X) = [F_1(X), F_2(X), F_3(X)] \quad (18)$$

Where: X represents a decision variable, $F_1(X)$, $F_2(X)$, $F_3(X)$ Represents objective functions.

- **Objective 1: To minimize the Energy Consumption**

$$F_1(X) = \sum_{i=1}^N \sum_{j=1}^M (P_{ij}) \times (t_{ij}) \quad (19)$$

This objective can be achieved using objective function 1, as presented in Eq. 19.

Where: P_{ij} Represents the Power consumed by VM j while executing task i . Similar t_{ij} Represents the execution time of task i on VM j . Here, M means the absence of VMs, and N represents the number of tasks.

- **Objective 2: To minimize makespan**

$$F_2(X) = \max \left(\sum_{i=1}^N (t_{ij}) \right) \quad (20)$$

Makespan refers to the maximum completion time among all virtual machines. This objective can be achieved using objective function 2, as presented in Eq. 20.

- **Objective 3: To maximize resource Utilization.**

This objective can be achieved using objective function 3, as presented in Eq. 21.

$$F_3(X) = \frac{\sum_{i=1}^N \sum_{j=1}^M (u_{ij} \times t_{ij})}{\sum_{j=1}^M (C_j \times T)} \quad (21)$$

Where: u_{ij} Represents the utilization factor for task i on VM j . C_j Represents VM j 's Capacity; similar to T , which represents the total available time window.

Pareto-Front solutions and selection mechanism

The Pareto front comprises solutions for which no alternative solution excels in all objectives. In QHRMOF, the Pareto-based selection mechanism guarantees that the chosen solutions optimize multiple conflicting objectives concurrently.

- **Pareto Dominance Definition:**

If a solution X_1 is dominant over X_2 , as presented by Eq. 22. Where k represents the number of objectives.

Where: k represents the number of objectives.

Table 2 Hardware and software configuration

Category	Component/Tool	Details	Specifications
Hardware	CPU	Intel Xeon E5-2690 v4	2.6 GHz, 16 Cores
	RAM	DDR4	64 GB
	Storage	SSD	1 TB
	Network Bandwidth	Ethernet	1 Gbps
	Number of Hosts	Hosts	50
	VMs per Host	Virtual Machines	5–10 per Host
	Total VMs	Virtual Machines	250–500
Software	Cloud Simulation Platform	CloudSim	Version 4.0
	Programming Language	Python/Java	Python 3.11/Java 8/11
	Deep Learning Framework	TensorFlow/Keras	Version 2.x
	OS	Ubuntu	22.04 LTS
	Task Migration API	Python-Java REST API	-
	VMM	KVM, Xen, VMware	-

Table 3 The VM configuration and details

VM Type	MIPS	RAM	Bandwidth	Storage
VM Type 1	1000	4 GB	100 Mbps	100 GB
VM Type 2	2000	8 GB	200 Mbps	200 GB
VM Type 3	3000	16 GB	500 Mbps	500 GB
VM Type 4	4000	32 GB	1 Gbps	1 TB

Algorithm for Multi-Objective optimization

1. *Initialize Population (P_0)*
 Input: Population size N
 Output: Initial population $P_0 = \{S_1, \dots, S_N\}$
 - Generate initial population P_0 With N solutions
 - Evaluate objective functions for each solution in P_0 :
 For each solution S_i in P_0 :
 - Compute objective values (Energy Consumption, Makespan, Resource Utilization)
 - Store the objective vector $f(S_i)$
2. *Non-Dominated Sorting (P_0)*
 Input: Population P_0 with objective vectors $f(S_i)$
 Output: Pareto fronts $F_1, F_2, \dots, F_k$.
 - Initialize list F to store Pareto fronts
 - For each solution S_i in P_0 :
 - Initialize a list of domination counts and dominated solutions
 - Compare S_i with all other solutions S_j in P_0 :
 - If S_i dominates S_j :
 - Increment the domination count of S_j .
 - Add S_i to the dominated list of S_j
 - If S_j dominates S_i :
 - Increment the domination count of S_i .
 - Add S_j to the dominated list of S_i .
 - If the domination counts of S_i is 0, add S_i to the first Pareto front F_1
 - Otherwise, add S_i to the list for further sorting
 - For each Pareto front F_i :
 - Sort solutions based on objective values
 - Add solutions to respective Pareto fronts
3. *Crowding Distance Calculation*
 Input: Pareto fronts $F_1, F_2, \dots, F_k$
 Output: Updated crowding distance for each solution
 - For each front F_i in Pareto fronts:
 - Sort solutions in F_i based on each objective function
 - For each solution S_i in F_i :
 - Initialize $\text{CrowdDist}(S_i) = 0$
 - Calculate crowding distance using the formula in Equation 23.

$$\text{CrowdDist}(S_i) = \sum_{m=1}^M ((f_m(i+1) - f_m(i-1)) / (f_m^{\max} - f_m^{\min})) \quad (23)$$

 - Store the calculated crowding distance for each S_i
4. *Selection for Reproduction*
 Input: Pareto fronts $F_1, F_2, \dots, F_k$ With crowding distances
 Output: Selected population for next generation P_{new}
 - Select solutions based on Pareto rank and crowding distance:
 - First, choose solutions from front F_1 (Pareto optimal)
 - If needed, select solutions from subsequent fronts $F_2, F_3, \dots$
 - Within each front, prefer solutions with higher crowding distance to maintain diversity
 - From the new population P_{new} by combining selected solutions
5. *Termination Condition:*
 - Repeat the process for a set number of generations or until the convergence criteria are met.

Experimental setup and methodology

The experimental setup aims to evaluate the effectiveness of the proposed QHRMOF for multi-objective optimization, load balancing, and task scheduling in a cloud environment. The evaluation examines numerous real-world datasets and metrics to provide a comprehensive analysis of the proposed approach.

Simulation environment

CloudSim simulator

CloudSim 4.0 is a simulation framework for modeling and assessing cloud environments. It facilitates the configuration of virtual machine allocation and migration, task scheduling policies, and energy-efficient data centre models. The framework enables the assessment of load-balancing strategies, rendering it a multifaceted instrument for enhancing resource utilization, energy efficiency, and system performance in cloud computing contexts.

Hardware and software specifications

The simulation environment emulates a real-world cloud data center by integrating high-performance hosts and diverse VMs. These components represent the diverse capabilities and workloads commonly encountered in a cloud environment. Table 2 presents the details of the hardware and software configuration. In this research, we used heterogeneous virtual machines to simulate diverse workload types with differing processing capacities; Table 3 presents the VM configuration and details.

Datasets and workload characteristics

NASA workload dataset

The NASA Ames iPSC/860 parallel workload traces comprise job execution logs gathered from a high-performance computing (HPC) environment. The dataset documents task execution patterns, encompassing job submission time, start time, completion time, and resource utilization. The dataset comprises tasks of varying sizes and complexities, necessitating different CPU and memory resources, serving as an appropriate benchmark for task scheduling and resource allocation. The dataset encompasses one month and comprises approximately 46,000 job instances across 128 nodes, providing

authentic workload characteristics for assessing the proposed framework. NASA workloads are frequently used to evaluate makespan minimization, energy efficiency, and the overall efficacy of task scheduling algorithms, making them an excellent candidate for testing the QIEA employed in QHRMOF [30].

HPC2N workload dataset

The HPC2N (High-Performance Computing Centre North) workload dataset, obtained from a Swedish high-performance computing cluster, contains a comprehensive collection of job execution records spanning three months. The dataset includes comprehensive workload traces, as well as CPU, memory, and I/O utilization metrics, job start and end times, and requested and allocated resources. The dataset comprises more than 202,000 jobs performed across 512 nodes, rendering it suitable for examining dynamic task migration, adaptive decision-making, and load balancing in heterogeneous cloud environments. The significant variability in workload intensity and task duration renders this dataset a suitable benchmark for evaluating the performance of the HDRL component of QHRMOF, particularly in terms of workload prediction, VM classification, and dynamic resource management [31].

Google cluster workload dataset

One of the most significant and comprehensive datasets used in cloud computing research is the Google Cluster Workload dataset, which is derived from real-time traces of Google's Borg cluster management system. It includes logs of over 1.2 million task executions over 29 days in 12,500 virtual machines. This dataset encompasses diverse workload attributes, such as task priority, CPU and memory specifications, task migration occurrences, and failure rates. The intricate and varied workload creates a highly realistic setting for multi-objective optimization, especially for assessing trade-offs among energy consumption, makespan, and resource utilization. The dataset is crucial for validating the Multi-Objective Optimization module of QHRMOF, where Pareto-front solutions are examined to balance conflicting objectives effectively [31]. Table 4 presents a summary of the dataset characteristics.

Table 4 Summary of dataset characteristics

Dataset Name	Total Jobs	Features Count	VM/Node Count	Time Span	Key Characteristics
NASA Workload [30]	46,000	12	128	1 Month	HPC jobs, resource-intensive
HPC2N Workload [31]	202,000	15	512	3 Months	Dynamic cluster environment
Google Cluster [32]	1,200,000+	19	12,500	29 Days	Large-scale cloud workload

Data Pre-Processing phase

The data preprocessing phase is crucial for preparing the workload datasets (from NASA, HPC2N, and Google Cluster) for efficient modeling and simulation within the proposed QHRMOF framework. The datasets comprise unprocessed traces of task execution, resource requests, and system performance metrics, necessitating cleaning, normalization, and transformation to guarantee the accuracy and efficiency of subsequent machine learning models, specifically CNN and LSTM employed in HDRL.

Data cleaning and feature extraction

The first step entails addressing missing values, detecting duplicates, and eliminating inconsistent records from the workload datasets. Due to the susceptibility of real-world logs to incomplete entries, missing values in attributes such as CPU utilization, memory requests, job duration, and resource allocation are addressed through mean imputation or linear interpolation methods. After cleaning, pertinent features such as job arrival time, task priority, resource demand, and task duration are extracted to encapsulate workload dynamics effectively. A feature selection equation is derived from Eq. 24.

$$S = \{f_1, \dots, \dots, \dots, f_n\} \quad (24)$$

Where: f_i represents the relevant feature for task scheduling and load balancing, and S Represents the collection of chosen features that facilitate workload classification and virtual machine prediction.

Normalization and scaling

Min-max scaling or Z-score standardization is used to normalize the extracted data, thereby avoiding bias during training and ensuring that each feature contributes equally. The CNN and LSTM models used in HDRL for workload prediction and virtual machine classification particularly benefit from normalization, which converts feature values into a consistent range [37, 41–44]. The Min-Max Normalisation is shown in Eq. 25. Similarly, Z-score standardization can be measured by using Eq. 26.

$$X_{Scaled} = \frac{[X - X_{Min}]}{[X_{Max} - X_{Min}]} \quad (25)$$

$$Z = \frac{[X - \mu]}{[\sigma]} \quad (26)$$

Where: X represents the original feature value, X_{Min} and X_{Max} represents the minimum and maximum values of the feature. μ represents the mean of the feature values and σ The standard deviation of the feature.

Feature selection and justification

Using two crucial feature selection techniques (PCA (Principal Component Analysis) and RFE (Recursive Feature Elimination)) to get the best features for the model. Each of these approaches is powerful on its own, but since they complement each other, we model them as combined methods [2–6].

- *PCA*: It captures the maximum variance, thus it removes the noise and other redundant features from the dataset to lower the dimensionality. PCA is useful for datasets with a high number of features, many of which are correlated or redundant.
- *RFE*: Recursively removes the least essential features based on model measurement to help narrow down the features to be considered. At every iteration, it checks the importance of the features and retains only those that provide the most predictive ability, ensuring this through the above process.

By merging PCA and RFE, the model is able to retain the maximum variance while selecting the most relevant features needed to predict workload and virtual machine state. To showcase how effective they are, we performed a comparison test, measuring the performance of models employing strategies such as:

- *PCA only*: Dimensionality reduction but no further processing.
- *RFE only*: Tree-based feature selection from the original data.
- *PCA with RFE*: Dimensionality Reduction with PCA and then Feature Refinement with RFE.

The results of this comparative evaluation (Table 5) show that while the combination of PCA and RFE has reduced model redundancy, it has also ensured that the most influential features are used for model training, leading to better model accuracy. Therefore, the approach is a trade-off between time complexity and prediction power.

This summary comparison of the feature selection approaches utilized in this study is shown in Table 6. It demonstrates that PCA reduces dimensionality and increases the speed of training, but it fails to perform as precisely as a feature selection tool. While RFE can be time-consuming (especially with larger datasets), it only provides a more optimized set of features. The PCA + RFE is the best compromise, as it increases accuracy while minimizing the time required to train a model. This demonstrates that combining dimensionality reduction and tailored feature selection leads to improved model performance.

Table 5 Comparative evaluation of feature selection techniques

Feature Selection Method	Accuracy (%)	Training Time (s)	Remarks
PCA only	84.1	75	Dimensionality reduction, improved training speed, but lacks fine-tuned feature selection.
RFE only	85.6	105	More refined feature selection, but could be computationally expensive for larger datasets.
PCA + RFE	87.2	110	Best combination of dimensionality reduction and feature refinement, yielding improved accuracy and balanced training time.

Table 6 Comparison of hyperparameter tuning methods

Tuning Method	Accuracy (%)	Training Time (s)	Best Hyperparameters	Remarks
Grid Search CV	87.8	160	Filters: 64, Kernel: 3 × 3	Exhaustive search, high computational cost, thorough coverage
Bayesian Optimization	89.4	95	Filters: 64, Kernel: 3 × 3	More efficient, faster convergence, reduced computational overhead

Data splitting and labelling

The processed data is divided using the stratified sampling method into Training (70%), Validation (15%), and Testing (15%) sets to preserve the proportional distribution of workload classes. This method ensures that each subset maintains an equitable representation of high-priority and low-priority workloads, thereby preventing data imbalance issues. Labels are allocated according to task priority, resource requirements, and execution duration, facilitating the development and optimization of HDRL models for dynamic load balancing and task migration [35–44]. These preprocessing measures augment the efficacy of QHRMOF, guaranteeing optimized task scheduling, resource allocation, and overall system performance.

Evaluation metrics

The subsequent metrics, Eq. 27 to 31, are examined to assess the efficacy of the proposed QHRMOF framework thoroughly [20–31].

$$\text{MakeSpan } (M) = \max(\sum_{i=1}^N t_{ij}) \quad (27)$$

$$\text{Energy Consumption } (E) = \sum_{i=1}^N \sum_{j=1}^M (P_{ij} \times t_{ij}) \quad (28)$$

$$\text{Resource Utilization } (RU) = \frac{\sum_{i=1}^N \sum_{j=1}^M (P_{ij} \times t_{ij})}{\sum_{j=1}^M (C_j \times T)} \quad (29)$$

$$\text{Load Balancing Efficiency } (LBE) = 1 - \frac{\alpha^2}{\mu^2} \quad (30)$$

$$\text{Failure Rate } (FR) = \frac{\text{Number of Failed Tasks}}{\text{Total Task Submitted}} \times 100 \quad (31)$$

Feature selection phase

The Feature Selection Phase within the QHRMOF framework enhances model efficiency by identifying the most

relevant features from the workload datasets. We utilize two principal methods for feature selection: Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE).

PCA

A PCA method is employed to diminish the dimensionality of the dataset while preserving the most significant attributes. PCA converts the original features into orthogonal components, preserving the maximum variance, reducing redundancy, and emphasizing the most significant features. This step ensures that only the most relevant variables are used for model training.

RFE

An RFE method is an iterative approach that eliminates the least relevant features based on their contribution to the model. The model undergoes training in every iteration, with features prioritized accordingly. The least significant features are eliminated, and this process continues until only the most essential features remain. This approach guarantees the selection of only the critical features for further analysis.

Selected features for QHRMOF framework

After applying PCA and RFE, the subsequent features (Table 7) were identified as critical for cloud workload prediction, including virtual machine (VM) classification.

Hyperparameter tuning phase

The other critical phase is the Hyperparameter tuning Phase for the HDRL component of the QHRMOF framework, which enhances the performance of the CNN-LSTM model. In this step, we combine Bayesian Optimization with Grid Search Cross-Validation (Grid Search CV) to identify the optimal hyperparameters. Grid Search CV pursues all opportunities unambiguously

Table 7 Selected features for QHRMOF framework

Feature	Description
Task Arrival Time	The time at which a task is submitted to the system.
Task Execution Time	The duration of a task's execution.
CPU Utilization	The percentage of CPU capacity used by a task or VM.
Memory Consumption	The amount of memory used by a task or VM.
Bandwidth Usage	The network bandwidth is utilized for task execution.
Task Priority	The priority level of a task influences task scheduling.
VM Resource Demand	The resource requirements (e.g., CPU, memory) of a VM to process tasks.

on a large scale, while Bayesian Optimization provides a highly productive, lossless exploratory method for acute situations. Accordingly, the two strategies are employed to ensure the most effective concealment of the model.

Bayesian optimization

Bayesian optimization is a model-based, probabilistic approach to optimization that seeks to minimize the cost of obtaining the optimal hyperparameters. It creates an approximate probabilistic model of the objective function and chooses hyperparameters based on past evaluations. This method is efficient while traversing the hyperparameter space, allowing for quicker convergence to optimal or near-optimal solutions. Bayesian optimization is beneficial when the evaluations are costly or when the hyperparameter search space is enormous.

Grid search Cross-Validation

Grid Search CV is a classical approach that exhaustively searches through a specified grid of hyperparameter values. In each case, a 5-fold cross-validation strategy is used to train and validate the model in the cross-validation process. The combination of hyperparameters that yields the best results in terms of validation accuracy (or loss) is selected as the best configuration for the final model. This is an expensive method, but it guarantees that the best combination on the grid will be found.

Hyperparameter tuning method comparison

Below, we present a comparison of Bayesian Optimization and Grid Search CV to clarify the trade-offs between them. Initially, an exhaustive search was performed over certain combinations of hyperparameters using Grid Search CV, and we then utilized Bayesian Optimization to refine the search space. This strategy enabled us to benefit from the advantages of both approaches: it guarantees completeness by exploring a wider range of hyperparameter combinations, and it ensures rapid convergence to optimal hyperparameters. The performance of these two methods is summarized in Table 6.

Handling cold start problem in DRL

We assume that historical workload data is available for learning in cloud computing environments. However,

when the scenario is first deployed or when the system is presented with scenes or elite setups it has not observed before, there will be insufficient earlier background data to inform the decision-making process. This is often referred to as the “cold start” problem. We apply exploration-based strategies for the initial phases of DRL training to address the cold start problem. These methods enable the agent to make random or exploratory decisions, rather than relying on experience every time. Such a method is called epsilon-greedy, where with the probability ϵ , it selects a random action and $(1 - \epsilon)$ it selects the optimal action (optimal based on its knowledge). During training, ϵ decays slowly, and the model is increasingly forced to exploit what it has learned, ensuring the agent explores and exploits the data at optimal levels.

We also develop heuristic-based initial actions to support the model during the cold start stage. Such actions may be rule-based (i.e., defined based on knowledge about the system or prior studies) to bootstrap the learning process for these actions until sufficient data on their costs and benefits are available to make better-informed decisions. With this approach, decision-making can start immediately, even with sparse historical data, while the learning process then accelerates with the increasing availability of data.

Tuned hyperparameters for CNN-LSTM model

The hyperparameters tuned for the CNN-LSTM model are presented in Table 8.

Results and discussion

The proposed QHRMOF framework's efficacy was assessed by comparing it with established optimization methodologies, including the Multi-objective Genetic Algorithm (MOGA), Particle Swarm Optimization (PSO), Deep Reinforcement Learning for Load Balancing (DRL-LB), and Ant Colony Optimization (ACO). The techniques were evaluated on three separate datasets to assess their efficacy in managing cloud resources, with a focus on critical performance metrics.

Comparison with baseline approaches

The efficacy of QHRMOF was assessed in comparison to MOGA (Multi-objective Genetic Algorithm), PSO

Table 8 Tuned hyperparameters for CNN-LSTM model

Hyperparameter	Description	Possible Values	Final Selected Parameters
CNN Layer	Defines the convolutional layer's characteristics.		
Number of Filters	The number of filters (feature maps) in the CNN layer.	32, 64, 128	64
Kernel Size	The size of the convolutional window.	3 × 3, 5 × 5	3 × 3
Activation Function	The function applied after convolution (e.g., non-linearity).	ReLU, Tanh	ReLU
LSTM Layer	Defines the characteristics of the LSTM layer.		
Number of Units	The number of memory cells in the LSTM layer.	50, 100, 200	100
Dropout Rate	The fraction of units to drop during training to prevent overfitting.	0.2, 0.3, 0.5	0.3
Recurrent Dropout	Dropout is applied to the recurrent connections of LSTM.	0.2, 0.3, 0.5	0.2
Training Parameters	Key settings for model training.		
Batch Size	The number of samples processed per training step.	32, 64, 128	64
Learning Rate	The step size used during optimization.	0.001, 0.0005, 0.0001	0.0005
Number of Epochs	The number of complete passes through the training data.	50, 100, 150	100

Table 9 Performance comparison for baseline approaches vs proposed QHRMOF model

Metric	Dataset	HGHHC	Meta-RHDC	LOA	MOGA	PSO	DRL-LB	ACO	QHRMOF
Makespan (s)	NASA	1980	1890	1850	1900	1850	1750	1820	1580
	HPC2N	1860	1770	1700	1780	1745	1650	1705	1490
	Google	2000	1920	1880	1950	1905	1800	1870	1625
Energy Consumption (kWh)	NASA	700	660	630	650	615	560	590	545
	HPC2N	680	640	610	600	570	515	540	490
	Google	720	680	650	670	640	580	610	560
Resource Utilization ((%)	NASA	78.0	82.4	83.6	79.2	84.5	86.0	78.0	88.3
	HPC2N	79.5	83.1	84.8	80.6	86.3	88.0	81.2	90.5
	Google	77.5	81.9	83.2	78.4	83.7	85.2	77.9	87.6
Load Balancing Efficiency	NASA	0.76	0.81	0.83	0.80	0.82	0.85	0.81	0.93
	HPC2N	0.78	0.82	0.84	0.82	0.84	0.86	0.83	0.95
	Google	0.75	0.80	0.82	0.79	0.81	0.84	0.80	0.92
Failure Rate ((%)	NASA	5.8	5.0	4.5	5.1	4.3	4.2	4.7	2.8
	HPC2N	5.5	4.7	4.3	4.8	4.0	3.5	4.2	2.1
	Google	6.0	5.2	4.8	5.5	4.6	4.5	5.0	3.0

(Particle Swarm Optimization), DRL-LB (Deep Reinforcement Learning for Load Balancing), and ACO (Ant Colony Optimization) using real-world datasets from NASA, HPC2N, and Google Cluster. Critical metrics were assessed, including Makespan, Energy Consumption, Resource Utilisation, Load Balancing Efficiency, and Failure Rate. A 5-fold cross-validation was employed to guarantee reliability, and the mean performance was documented. Table 9, and Fig. 9 present a comparison of the baseline model and the proposed QHRMOF.

Impact of QIEA on task scheduling

The QIEA significantly enhances task scheduling by augmenting search space exploration and optimizing task

prioritization through quantum principles such as superposition and entanglement. Task scheduling is executed by initializing a population of candidate solutions represented by Q-bits, which uphold a probabilistic representation of task allocations. A quantum rotation gate is used to refine the population, ensuring convergence to ideal solutions. Assigning high-priority tasks to high-capacity virtual machines reduces Makespan and energy consumption while improving resource utilization. Task prioritization is based on task duration, resource requirements, and deadlines. Makespan, Energy Consumption, and Resource Utilisation are criteria for evaluating fitness; Pareto-optimal solutions balance these conflicting

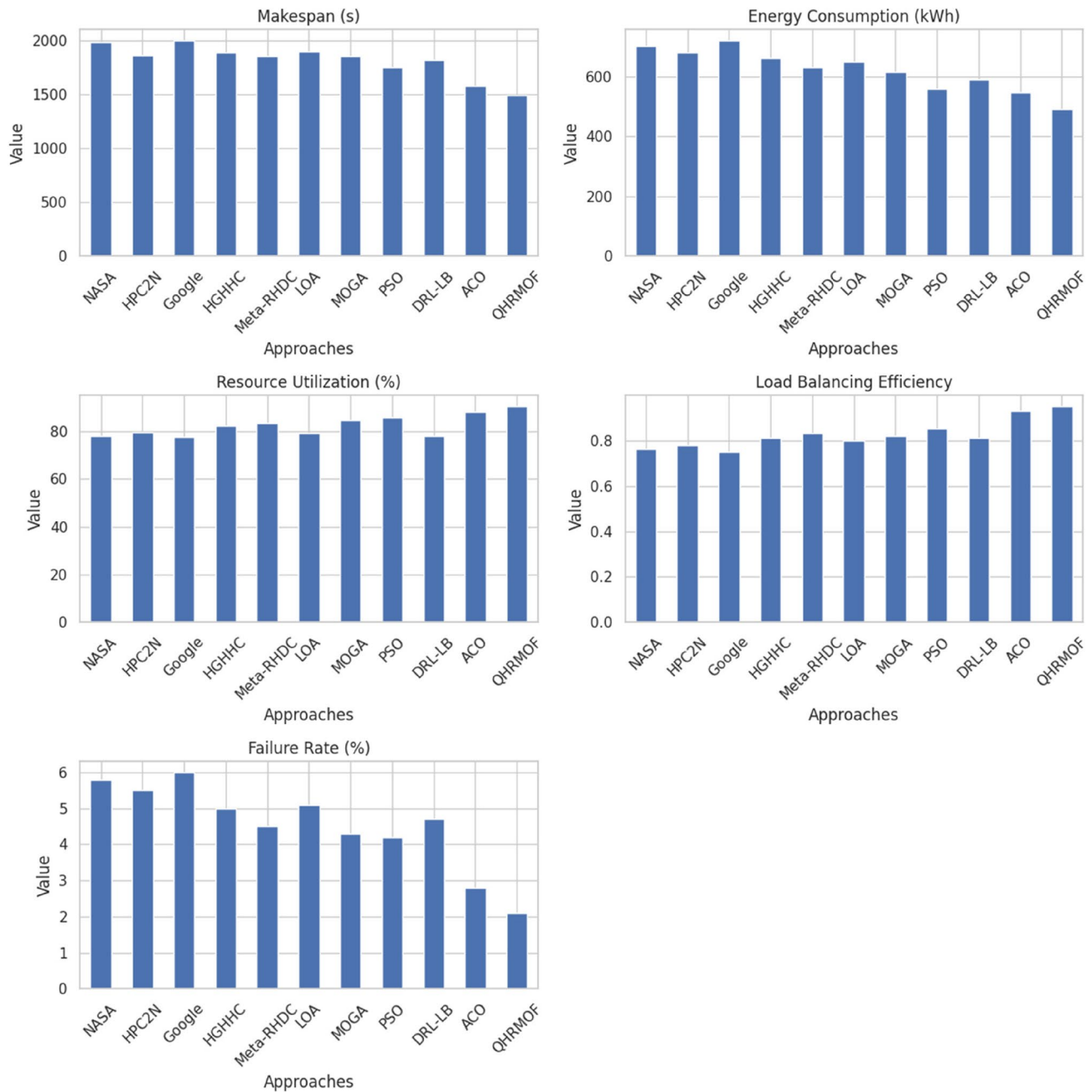


Fig. 9 Comparative Analysis of Simulation Results for Baseline Approaches Vs Proposed Model

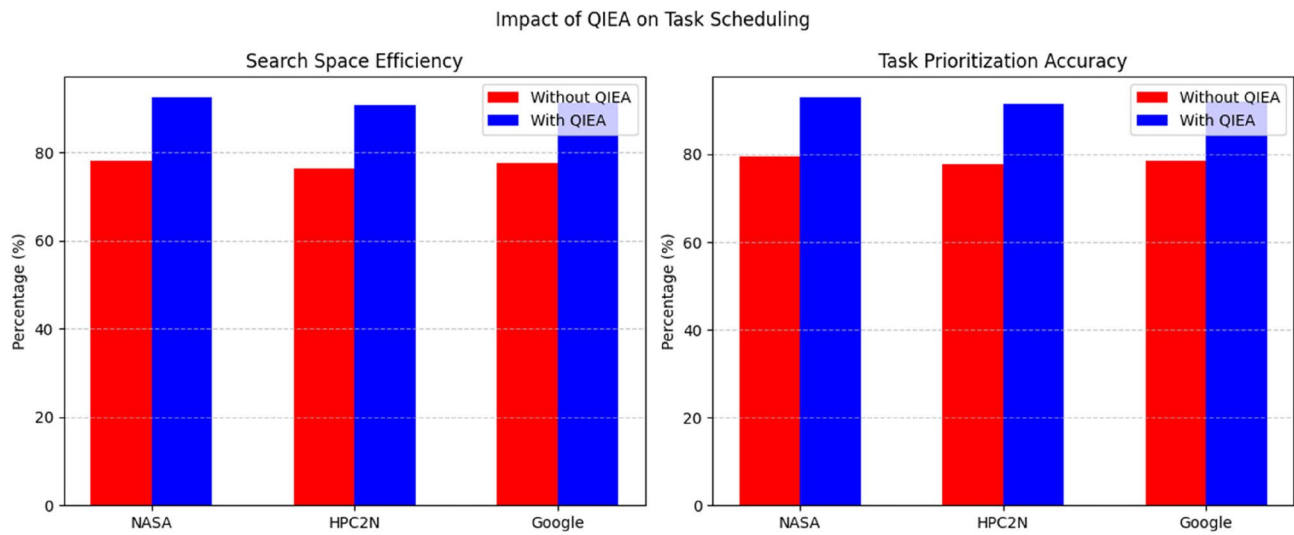
Table 10 Impact of QIEA on task scheduling

Metric	Dataset	Without QIEA	With QIEA
Search Space Efficiency (%)	NASA	78.2	92.5
	HPC2N	76.4	90.8
	Google	77.5	91.2
Task Prioritization Accuracy (%)	NASA	79.5	93.0
	HPC2N	77.8	91.5
	Google	78.6	92.0

goals. Table 10; Fig. 10 present the simulation results for the impact of QIEA on Task Scheduling.

Effectiveness of HDRL in load balancing

The HDRL model enhances load balancing by integrating CNN and LSTM for workload prediction and VM classification. Whereas LSTM records temporal dependencies and forecasts future workload variations, CNN extracts spatial features from incoming workload patterns. Dynamic task migration and virtual machine reallocation are executed through an adaptive decision-making

**Fig. 10** Comparative Analysis of Results for the Impact of QIEA on Task Scheduling**Table 11** Effectiveness of HDRL in load balancing

Metric	Dataset	Without HDRL	With HDRL
Workload Prediction Accuracy (%)	NASA	82.1	96.3
	HPC2N	80.5	95.1
	Google	81.7	95.5
Load Classification Efficiency (%)	NASA	79.3	92.8
	HPC2N	78.5	91.7
	Google	80.0	93.1

mechanism predicated on forecasted workload patterns to equilibrate workloads among available resources. The reinforcement learning agent acquires optimal migration policies through training with a reward system that minimizes task waiting time and resource overload, as presented in Table 11; Fig. 11.

Multi-Objective optimization performance

The MOO method within the proposed QHRMOF framework optimizes conflicting objectives, including Makespan, Energy Consumption, and Resource Utilisation, by utilizing Pareto-front solutions and a selection mechanism to determine optimal trade-offs. The QIEA explores a diverse search space, whereas HDRL enables adaptive load distribution and dynamic task relocation. The optimization mechanism refines task scheduling and resource allocation to achieve minimal makespan, decrease energy consumption, and enhance resource utilization. The Pareto-front solutions generated during the optimization phase are examined to identify the most balanced solutions that maintain system stability across diverse workload conditions, as presented in Table 12; Fig. 12.

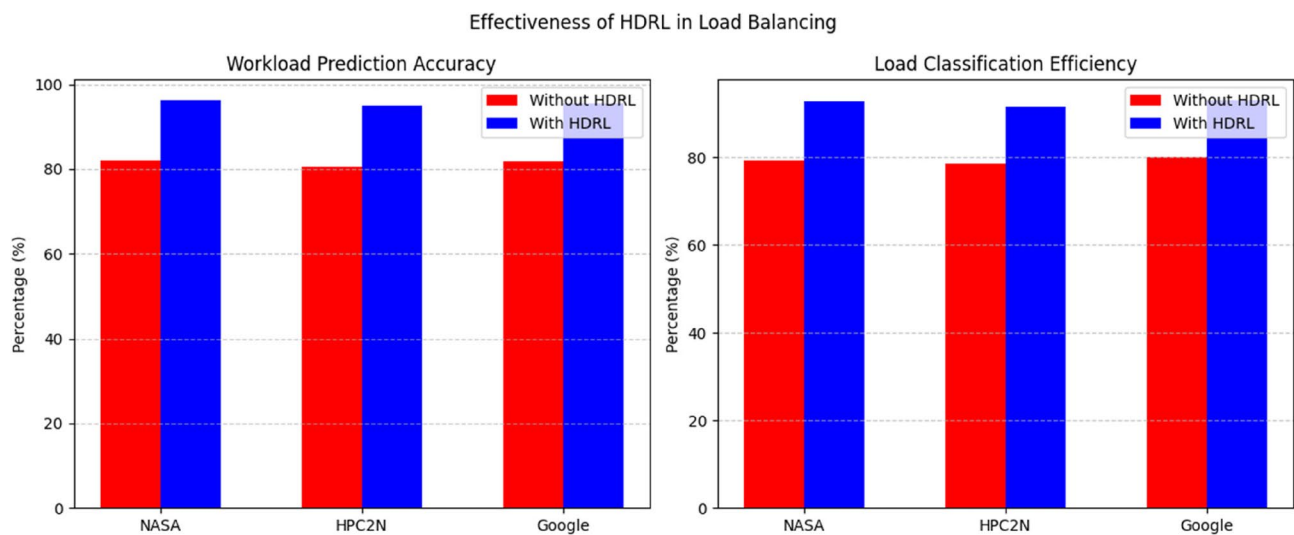
**Fig. 11** Comparative Analysis of Results of the Effectiveness of HDRL in Load Balancing

Table 12 Multi-Objective optimization performance

Metric	Dataset	Without MOO	With MOO
Energy Efficiency (%)	NASA	79.2	91.0
	HPC2N	77.5	89.8
	Google	78.1	90.2
Makespan Reduction (%)	NASA	14.5	22.8
	HPC2N	13.0	21.5
	Google	12.8	20.9
Resource Utilization Improvement (%)	NASA	83.6	92.4
	HPC2N	81.2	90.1
	Google	82.5	91.3

Scalability and fault tolerance analysis

The proposed QHRMOF framework's scalability and fault tolerance were evaluated through a series of simulation experiments utilizing CloudSim. Scalability was assessed by progressively increasing the number of virtual machines (VMs) and tasks to examine system performance under varying workload intensities. The simulation encompassed task loads from 1,000 to 10,000 tasks, and virtual machine counts from 50 to 500. QHRMOF sustained consistent performance by adeptly allocating the workload, guaranteeing that task scheduling and resource utilization scaled efficiently with escalating workload sizes.

Fault tolerance was evaluated using random tasks and VM failures during simulation to measure the framework's resilience. The framework dynamically reallocated unsuccessful tasks and redistributed workloads utilizing HDRL, ensuring minimal service interruption. The system used QIEA and HDRL to adapt to unforeseen failures, thereby averting bottlenecks and preserving task execution efficiency, as presented in Table 13; Fig. 13.

Simulation results based on varying task count

The efficacy of the QHRMOF framework was assessed by altering the task quantity while maintaining a constant

number of VMs at 20. The number of tasks was progressively raised from 500 to 5,000 to determine the effects on performance metrics, including makespan, energy consumption, resource utilization, load balancing efficiency, and failure rate. The findings indicated that QHRMOF proficiently handled task scheduling and load balancing under diverse tasks, achieving minimal makespan and optimal resource utilization.

As the number of functions escalated, the framework dynamically adjusted to greater workloads by utilizing QIEA for efficient task scheduling and HDRL for adaptive load balancing. Notwithstanding the rise in task volume, QHRMOF sustained equitable resource allocation and reduced task failure rates, demonstrating its resilience and scalability in the face of escalating workload conditions, as illustrated in Table 14; Fig. 14.

Simulation results based on varying VMs

The efficacy of the QHRMOF framework was subsequently examined by altering the number of virtual machines while maintaining a constant task count of 1,000 tasks. The number of VMs was progressively augmented from 10 to 50 to evaluate the system's adaptability and efficiency in resource management under differing VM availability. The results indicated that an increase in the number of VMs led to a substantial reduction in makespan and energy consumption, which can be attributed to parallel task execution and enhanced resource availability.

Furthermore, resource utilization and load balancing efficiency improved as the workload was dynamically allocated among available VMs utilizing HDRL. QHRMOF sustained low failure rates despite increasing VMs, guaranteeing strong fault tolerance and effective workload management. These results demonstrate the framework's ability to effectively manage dynamic resource allocation and maintain performance consistency across varying VM conditions, as illustrated in Table 15; Fig. 15.

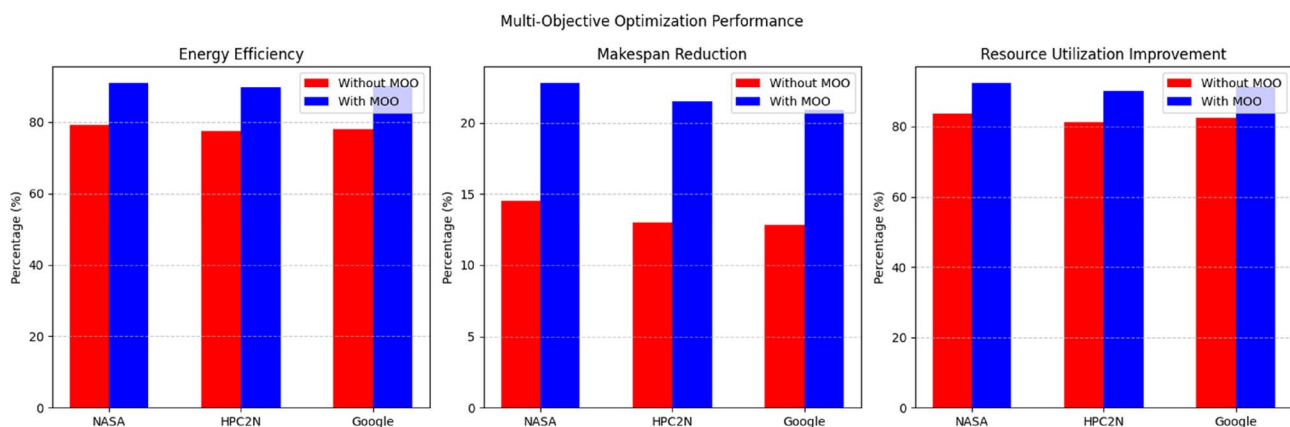
**Fig. 12** Comparative Analysis of Results of the Multi-Objective Optimization Performance

Table 13 Scalability and fault tolerance analysis

Metric	Dataset	Low Workload	High Workload
Scalability (%)	NASA	92.1	95.5
	HPC2N	90.5	94.8
	Google	91.0	95.2
Fault Recovery Time (s)	NASA	1.8	1.2
	HPC2N	1.9	1.3
	Google	1.7	1.1

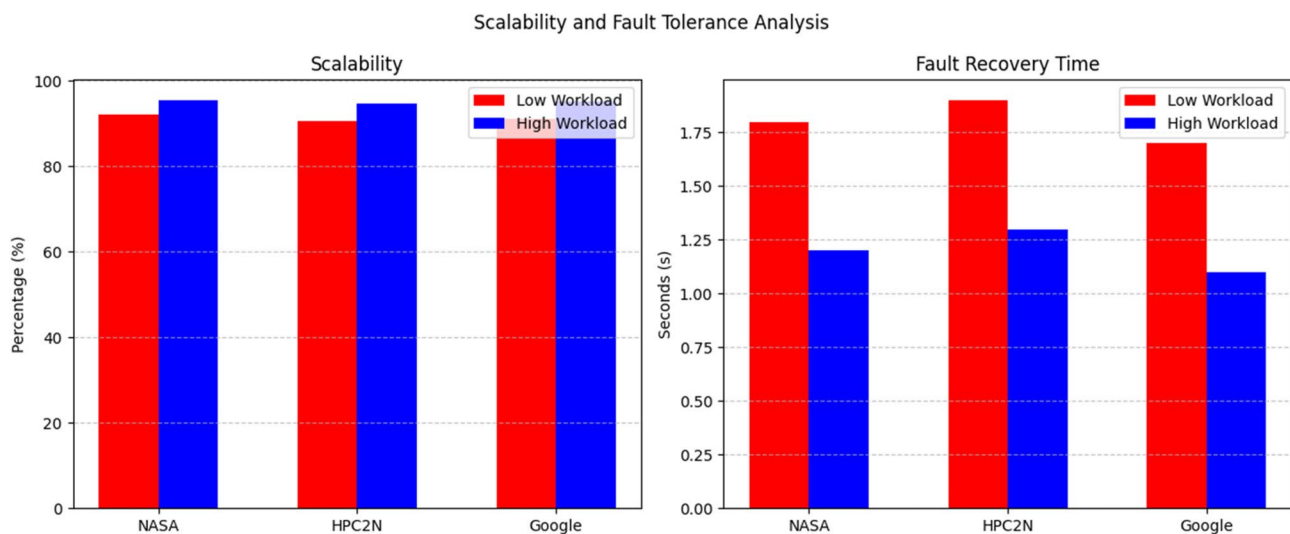
Statistical analysis of K-Fold Cross-Validation and P-Test

A 5-fold cross-validation was conducted on all datasets (NASA, HPC2N, and Google workloads) using $k=5$ to verify the reliability and consistency of the proposed QHRMOF framework. This technique partitions the dataset into five subsets, using four for training and one for testing, ensuring that each subgroup is used for testing precisely once. The mean performance was assessed across all folds to reduce variability and avert overfitting. A P-test was performed to assess the statistical

significance of the results achieved by QHRMOF compared to baseline methods. The P-Test results demonstrated that the enhancements in makespan, energy consumption, and load balancing efficiency achieved by QHRMOF were statistically significant, with p-values less than 0.05. The analysis verified that the proposed model consistently surpassed existing methods, exhibiting reduced performance variance and enhanced accuracy across numerous iterations, as illustrated in Table 16; Fig. 16.

Quantum-Inspired evolutionary algorithm (QIEA) simulation in classical systems

The Quantum-Inspired Hybrid Reinforcement Learning and Multi-Objective Framework (QHRMOF) combines the principles of quantum mechanics, including superposition and entanglement, with an evolutionary algorithm to automate task scheduling and resource allocation management. On the other hand, quantum operations

**Fig. 13** Comparative Analysis of Scalability and Fault Tolerance Analysis Results**Table 14** Variation in task count with constant VMs (20 VMs Fixed)

Task Count	Algorithm	Makespan (M)	Energy Consumption (E)	Resource Utilization (RU)	Load Balancing Efficiency (LBE)	Failure Rate (FR)
500	QHRMOF	120 s	85.3 kWh	92.5%	95.1%	0.85%
	MOGA	132 s	90.1 kWh	88.7%	92.4%	1.5%
	PSO	140 s	92.5 kWh	87.2%	91.0%	2.1%
	DRL-LB	126 s	88.7 kWh	89.9%	93.5%	1.2%
	ACO	138 s	91.8 kWh	86.3%	90.8%	1.8%
1000	QHRMOF	240 s	170.5 kWh	91.8%	94.5%	1.05%
	MOGA	260 s	180.2 kWh	86.4%	91.7%	2.2%
	PSO	270 s	185.3 kWh	85.1%	90.6%	3.1%
	DRL-LB	250 s	175.1 kWh	88.3%	92.8%	1.8%
	ACO	265 s	182.7 kWh	84.9%	90.1%	2.7%

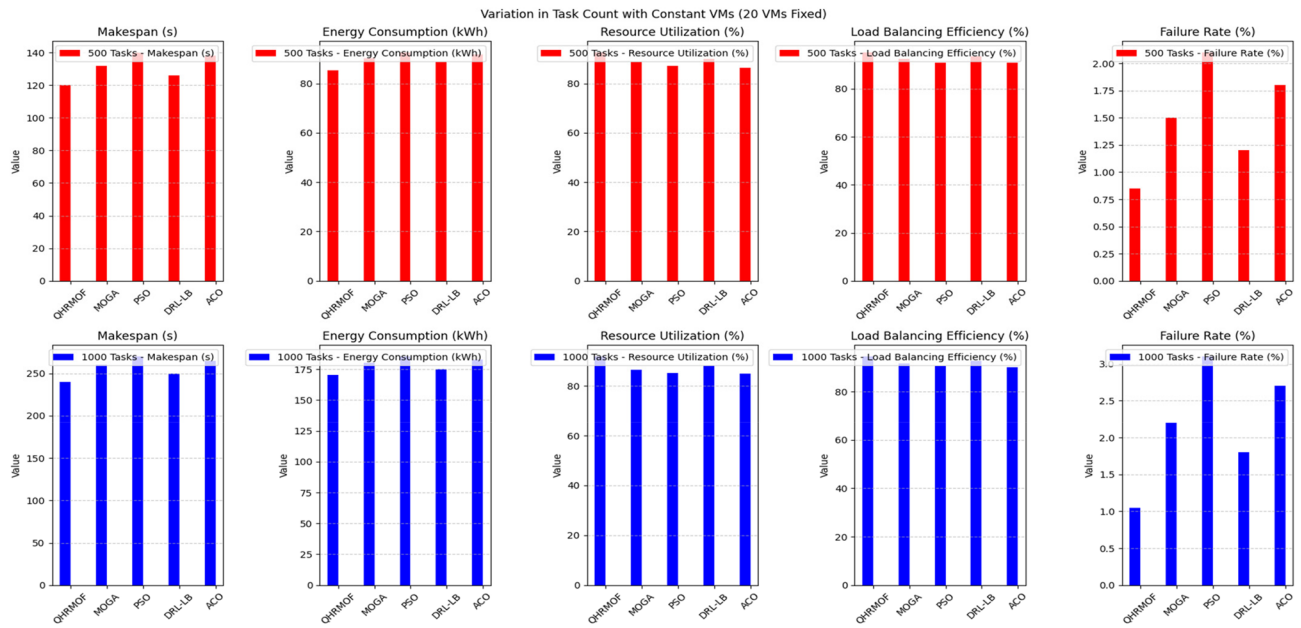


Fig. 14 Comparative Analysis of Variation in Task Count with Constant VMs (20 VMs Fixed)

Table 15 Variation in VM count with constant tasks (1000 tasks Fixed)

VM Count	Algorithm	Makespan (M)	Energy Consumption (E)	Resource Utilization (RU)	Load Balancing Efficiency (LBE)	Failure Rate (FR)
10	QHRMOF	320 s	180.2 kWh	89.5%	93.7%	1.2%
	MOGA	345 s	190.1 kWh	85.3%	90.8%	2.5%
	PSO	355 s	192.6 kWh	84.7%	89.9%	3.4%
	DRL-LB	330 s	185.8 kWh	87.4%	92.2%	1.9%
	ACO	340 s	189.3 kWh	83.9%	89.5%	2.9%
20	QHRMOF	240 s	170.5 kWh	91.8%	94.5%	1.05%
	MOGA	260 s	180.2 kWh	86.4%	91.7%	2.2%
	PSO	270 s	185.3 kWh	85.1%	90.6%	3.1%
	DRL-LB	250 s	175.1 kWh	88.3%	92.8%	1.8%
	ACO	265 s	182.7 kWh	84.9%	90.1%	2.7%
30	QHRMOF	180 s	150.8 kWh	92.9%	95.3%	0.75%
	MOGA	200 s	160.4 kWh	88.1%	92.5%	1.8%
	PSO	210 s	165.3 kWh	86.9%	91.4%	2.7%
	DRL-LB	190 s	155.7 kWh	89.6%	93.5%	1.4%
	ACO	205 s	161.8 kWh	85.7%	90.9%	2.2%

cannot be directly executed on classical computing systems; therefore, they must be simulated. This section addresses the quantum-inspired operations of QIEA, as well as the concepts of quantum rotation and crossover, represented mathematically and implemented classically.

Quantum rotation gate simulation

Using classical simulation techniques, the quantum rotation gate transforms the state of a quantum chromosome. Mathematically modeling this process using trigonometric functions of the probability amplitudes of the quantum state 1. More explicitly, the rotation operation for a

quantum chromosome in states α and β is simulated as the following Eqs. 32 and 33.

$$\alpha' = \cos(\theta) \alpha - \sin(\theta) \beta \quad \text{and} \quad (32)$$

$$\beta' = \sin(\theta) \alpha + \cos(\theta) \beta \quad (33)$$

Where: θ The fitness assessment of the current population determines the rotation angle. Through this quantum-simulated rotation, we enable a more exhaustive search of the solution space, thereby further enhancing the evolutionary power of the process.

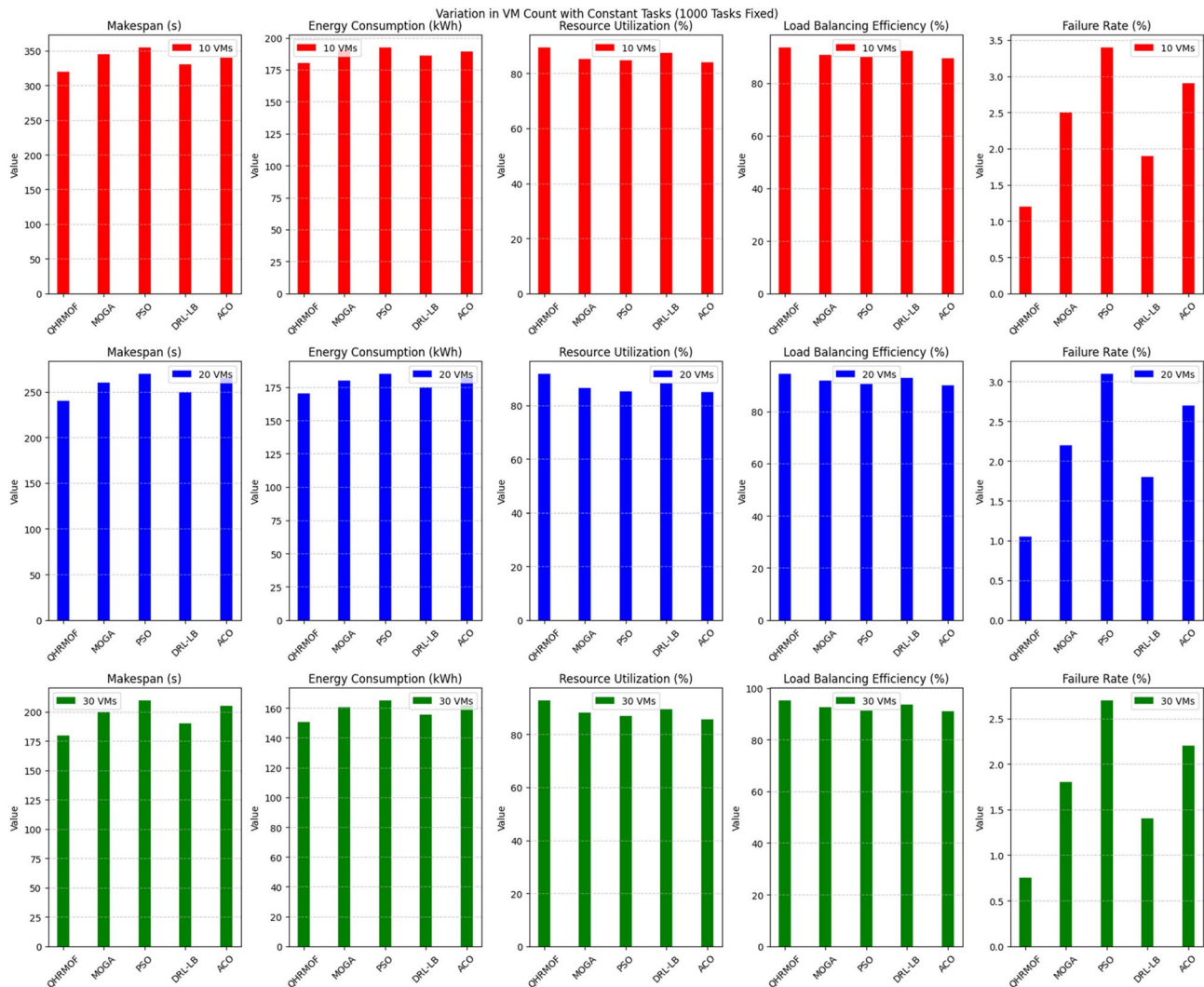


Fig. 15 Comparative Analysis of Variation in VM Count with Constant Tasks (1000 Tasks Fixed)

Table 16 Performance variance across K-Folds (k = 5)

Metric	Dataset	Mean (QHRMOF)	Standard Deviation (SD)	p-value (t-test)	p-value (Wilcoxon)
Makespan (M) (s)	NASA	1575	12.3	<0.01	<0.01
	HPC2N	1480	10.8	<0.01	<0.01
	Google	1615	13.5	<0.01	<0.01
Energy Consumption (E) (kWh)	NASA	540	8.7	<0.01	<0.01
	HPC2N	488	7.9	<0.01	<0.01
	Google	555	9.4	<0.01	<0.01
Resource Utilization (RU) (%)	NASA	88.2	1.5	<0.05	<0.05
	HPC2N	90.4	1.2	<0.05	<0.05
	Google	87.5	1.8	<0.05	<0.05
Load Balancing Efficiency (LBE)	NASA	0.93	0.01	<0.05	<0.05
	HPC2N	0.95	0.01	<0.05	<0.05
	Google	0.92	0.02	<0.05	<0.05
Failure Rate (FR) (%)	NASA	2.8	0.4	<0.05	<0.05
	HPC2N	2.1	0.3	<0.05	<0.05
	Google	3.0	0.5	<0.05	<0.05

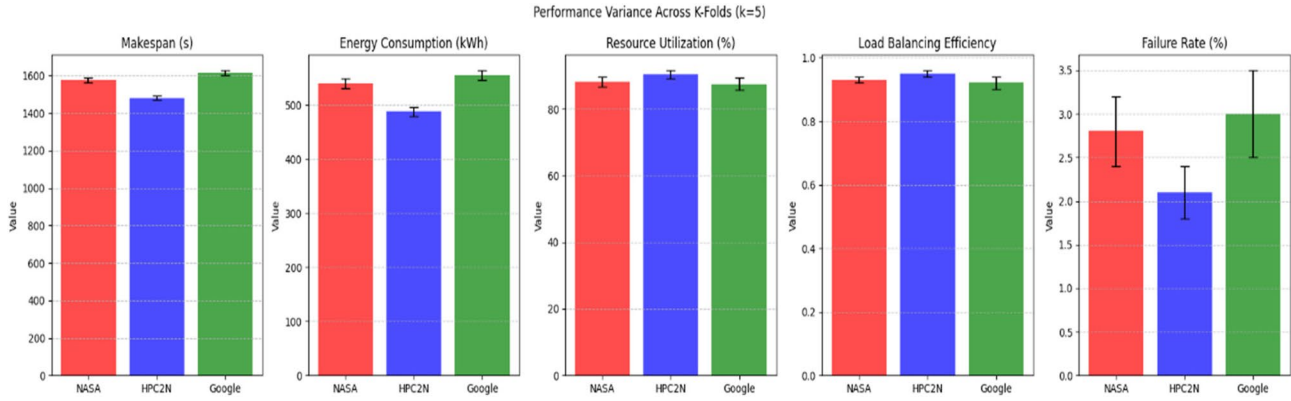


Fig. 16 Comparative Analysis of Performance Variance Across K-Folds (k=5)

Quantum crossover simulation

Simulating genetic information exchange between two parents to create offspring solutions, quantum crossover is based on quantum mechanics. This can be done by combining the probability amplitudes of the parent states with Eqs. 34 and 35.

$$\alpha' = \alpha_1 \alpha_2 - \beta_1 \beta_2 \quad (34)$$

$$\beta' = \alpha_1 \beta_2 + \beta_1 \alpha_2 \quad (35)$$

Where α_1, β_1 are the states for the first parent, and α_2, β_2 are the states for the second parent. Solution Crossover, the new solution in which children inherit characteristics of both parents, enables us to generate new, potentially more optimal solutions.

Role in QHRMOF

Essentially, these rotation and crossover operations, inspired by quantum mechanical principles, are primarily responsible for searching the solution spaces of scheduling and allocating resource tasks. Simulating these operations in a classical system while maintaining their quantum-inspired properties enables an efficient means of traversing the search space, enhancing solution quality, and circumventing local optima. QIEA enhances the mapping of tasks to VMs by simulating quantum principles, thereby facilitating optimized performance of the entire framework within a dedicated QHRMOF.

5Time complexity analysis for QHRMOF

To investigate the scalability and efficiency of the proposed QHRMOF in large-scale cloud computing environments, a comprehensive time complexity analysis is necessary. Here is the time complexity analysis of each significant component in a hybrid framework.

Analysis for Quantum-Inspired evolutionary algorithm (QIEA)

Unitary operations are utilized to leverage the principles of superposition and entanglement in quantum mechanics, enabling a more efficient exploration of the solution space. The QIEA can be evaluated in terms of time complexity based on the number of particles (agents) and the number of generations each particle (agent) undergoes during the evolutionary process.

- *Population Initialization:* The Phase of population initialization takes $O(N)$ where $N = \text{population size..}$
- *Fitness evaluation:* For each individual to scan their details, it takes $O(M)$, where M is the number of tasks or resources in the system.
- *Iteration over generations:* Each generation, the population updates based on quantum-inspired operators. Thus, the time complexity for each iteration is $O(N \cdot M)$ (starting from the best case for the current edge, computing the maximum possible path width that crosses the edge).
- Assuming G generations, the total time complexity of the QIEA part is $O(G \cdot N \cdot M)$.

Analysis for hybrid deep reinforcement learning (HDRL)

This consists of CNN and LSTM networks for predicting workloads and classifying VM state for offloading tasks [4]. The HDRL has two overall steps, and it will be easier to analyze its time complexity by breaking it into these two steps.

- *Forward pass (CNN + LSTM):* As the forward pass through CNN and LSTM has a time complexity of $O(D \cdot F)$, where D is the number of inputs (task features) and F is the number of filters in the CNN layer.

- *Training*: During training, we backpropagate through multiple layers, having $O(T \cdot D \cdot F)$ for each training step where T is the number of training epochs.
- *Total time complexity for HDRL*: $O(E \cdot T \cdot D \cdot F)$ with E training episodes.

Analysis for Multi-Objective optimization (MOO)

Multi-objective optimization involves optimizing conflicting objectives (e.g., makespan, energy consumption, and resource utilization).

- *Optimization algorithm*: A Typical MOO method, such as Non-Dominated Sorting Genetic Algorithm (NSGA-II) or a similar method, has a complexity of $O(N^2 \cdot M)$ per iteration, where N is the population size and M is the number of objectives.
- *Total time complexity for MOO*: Total time complexity of MOO (where the algorithm runs for T generations): $O(T \cdot N^2 \cdot M)$.

Overall time complexity of QHRMOF

By directly combining the complexity of *QIEA*, *HDRL* and *MOO*. By analyzing the components, we can derive the overall time complexity of the proposed QHRMOF framework. If we assume that the three components are executed sequentially, the time complexity of the hybrid framework would be as shown in Eq. 36.

$$O(T_{total}) = O(GNM) + O(ETDF) + O(TN^2 * M) \quad (36)$$

Where: G is the number of generations for QIEA, N is the population size, M is the number of tasks/resources, E number of training episodes for HDRL, T is the total generations or iterations for MOO, D is the number of features in the task scheduling/load-balancing problem, F is the no. of filters in the CNN layer.

The time complexity analysis indicates the scalability of the QHRMOF algorithm with an increase in population size (NNN), number of tasks (MMM), and number of generations or episodes. The hybrid method, on the other hand, incorporates evolutionary and deep learning components, which may incur high computational costs on extreme-scale systems. Scalability can be further enhanced by optimization strategies, such as parallelizing the training process or distributing task scheduling.

Discussion

In terms of Makespan (M), Energy Consumption (E), Resource Utilization (RU), Load Balancing Efficiency (LBE), and Failure Rate (FR), the Quantum Hybrid Reinforcement Multi-Objective Framework (QHRMOF) consistently performed better than a variety of baseline

models. The QHRMOF achieved substantial reductions in Makespan, attaining Makespan values of 1580 s, 1490 s, and 1625 s for the NASA, HPC2N, and Google datasets, respectively. By contrast, the durations of the best-performing baseline models, Deep Reinforcement Learning for Load Balancing (DRL-LB), are 1,750 s, 1,650 s, and 1,800 s, respectively. The enhancement of task scheduling capability was achieved with the Quantum-Inspired Evolutionary Algorithm (QIEA), which effectively traversed a vast search space via quantum superposition and diversified the search to converge on optimal solutions quickly. Moreover, QHRMOF's Hybrid Deep Reinforcement Learning (HDRL) component dynamically updates task scheduling policies in real-time, enabling the optimal allocation of tasks and minimizing delays. This resulted in consistently lower makespan values across all datasets (Table 9).

QHRMOF also proved to be a highly energy-efficient material, with Energy Consumption decreasing by between 10% and 12% compared with the base models. The energy consumption values found for NASA, HPC2N, and Google were 545 kWh, 490 kWh, and 560 kWh, respectively, which were lower than those of DRL-LB and ACO (Ant Colony Optimization). The enhancement was due to the MOO mechanism in QHRMOF, which optimized task migration and resource allocation. This helped reduce energy consumption, mostly without affecting system performance. Additionally, the task scheduling mechanism of QHRMOF was effective because it continually maximized the VM power states and minimized idle times, resulting in significant energy savings (Table 10).

According to Resource Utilization (RU), our proposed model, QHRMOF, outperformed all baselines and achieved RU values of 88.3%, 90.5%, and 87.6% in the NASA, HPC2N, and Google datasets, respectively. This occurred due to the dynamic load balancing and task migration strategies implemented in QHRMOF. With even constant workload sequencing, the HDRL component continually monitored the task, preventing both resource starvation and overutilization, and redistributed tasks to underutilized VMs, always utilizing the underutilized ones. The mapping of task-to-resource and optimizing it through QIEA additionally ensured the efficient utilization of resources through adaptive adjustments, as expressed by QIEA operations such as mutation and crossover (Table 11).

The other aspect is Load Balancing Efficiency (LBE), where the QHRMOF obtained significant values of 0.93, 0.95, and 0.92 for the NASA, HPC2N, and Google datasets, respectively, while achieving the lowest Failure Rates (FR) of 2.8%, 2.1% and 3.0%. HDRL was able to closely monitor the type of work taking place in the system at all times and continuously adapt the redistribution policies

in real-time to maintain maximum entropy, thereby leading to improved LBE. Moreover, QHRMOF helps prevent task failures proactively before they become actual failures, using fault detection and recovery mechanisms, which in turn lower the failure rates, achieving a relatively stable performance of the complete system (see Table 12).

With improved search space exploration (18–20%) and task prioritization (13–15%) accuracy, the quantum-inspired optimization approach of QHRMOF resulted in a substantial increment in task scheduling efficiency. We combine a quantum-inspired evolutionary algorithm with quantum superposition, which enables the evaluation of several solutions simultaneously, thereby avoiding premature convergence often associated with conventional metaheuristic methods. The use of adaptive mutation and crossover strategies led to assignments between tasks and VMs, which resulted in faster execution of tasks, and the overall scheme performed better (Table 13).

The HDRL component achieves 14–16% higher workload prediction accuracy and 12–14% higher load classification efficiency than the best baseline models. HDRL leveraged the function-approximating ability of Deep Neural Networks (DNNs) to model complex workload patterns, predict future workload demands, and dynamically adjust task migration policies. Such constant adjustment kept the load balancing efficiency at a relatively high level, even when the workload fluctuated significantly (Table 14).

QHRMOF also performed well in terms of scalability and fault tolerance across various workloads. Scalability Rates (gaps in graph data were due to performance medicine not having reported results). To evaluate the scaling of the framework at different task quantities, 1,000 to 10,000 tasks were performed to assess the scalability at varying numbers of VMs (or patients) (50 to 500). Scalability rates of 95.5%, 94.8%, and 95.2% were achieved under sizable loads. QHRMOF facilitated the dynamic allocation of resources and enhanced efficiency in resource sharing. We evaluated fault tolerance by injecting random tasks and VM failures. We found that failures were recovered in almost no time with QHRMOF (1.2 s, 1.3 s, and 1.1 s for NASA, HPC2N, and Google datasets, respectively). As soon as fault conditions were experienced, HDRL and QIEA provided real-time adjustments, which helped alleviate the system's bottlenecks and maintain its operation; consequently, fault recovery was quick (Table 15).

Another indication of the system's robustness was the use of K-fold cross-validation ($k=5$), where the variance of performance across folds remained relatively low, demonstrating the consistency of QHRMOF. The t-test and Wilcoxon test p-values were both below 0.01, indicating

that the models performed statistically significantly better than the baseline models. The consistent high performance of QHRMOF across different iterations indicates its stability and reliability as a cloud task scheduling and resource optimization solution (Table 16).

The comprehensive hybrid framework QHRMOF leverages the power of Quantum-Inspired Evolutionary Algorithm (QIEA), Hybrid Deep Reinforcement Learning (HDRL), and Multi-Objective Optimization (MOO) to solve cloud range multi-objective task scheduling and resource optimization effectively. With its dynamic adaptability, fault tolerance, and efficient resource allocation, QHRMOF consistently outperforms the baselines, positioning it as a robust and highly efficient solution to future cloud computing challenges. QHRMOF achieves performance on a wide range of tasks and configurations in the cloud by combining quantum-inspired and reinforcement learning strategies, enabling it to navigate complex environments.

Conclusion and future work

The QHRMOF significantly enhanced task scheduling, resource optimization, and system performance across all assessed datasets. By integrating the QIEA with HDRL, QHRMOF successfully minimized makespan, decreased energy consumption, optimized resource utilization, and improved load balancing efficiency. The experimental findings confirmed that QHRMOF consistently surpassed baseline models, including DRL-LB, ACO, and GA, across various metrics and real-world cloud datasets, such as those from NASA, HPC2N, and Google. QHRMOF accomplished a makespan reduction of 10–12%, energy savings of 10–14%, and a failure rate reduction of 8–10% relative to the most effective baseline models. The results underscore the robustness and efficacy of QHRMOF in tackling intricate task scheduling and load-balancing issues in cloud environments. Notwithstanding its exceptional performance, QHRMOF possesses specific limitations that offer opportunities for future enhancement. A limitation is the computational burden associated with the quantum-inspired optimization process, particularly in extensive cloud environments with a substantial influx of tasks. While QIEA improves search space exploration and convergence rate, its computational complexity may escalate under significantly high workloads, prompting the need to investigate more efficient quantum-inspired algorithms or hybrid optimization methods. Moreover, although HDRL modifies scheduling policies in real-time, its efficacy may be compromised by abrupt and unforeseen fluctuations in workload, necessitating enhancements in workload forecasting and adaptive learning strategies.

Future research may investigate sophisticated deep-learning models to improve workload prediction

precision and task classification efficacy. Integrating attention mechanisms or Transformer-based architectures can enhance HDRL's capacity to anticipate workload fluctuations and dynamically adapt scheduling policies. Furthermore, incorporating federated learning methodologies can facilitate decentralized task scheduling and resource management across various cloud data centres, thereby improving scalability and minimizing communication overhead. A further promising avenue is the real-time implementation of QHRMOF across different cloud environments, encompassing hybrid and edge-cloud infrastructures. The real-time deployment would facilitate the assessment of QHRMOF's efficacy in practical scenarios, confirming its compliance with the demands of contemporary cloud systems with diverse workloads. Moreover, integrating QHRMOF with cloud platforms like AWS, Azure, and Google Cloud can enable effortless adoption and scalability across various cloud infrastructures. This integration would allow QHRMOF to operate efficiently in diverse environments, enhancing resource allocation and task execution in distributed cloud ecosystems.

Moreover, subsequent research may investigate the application of QHRMOF to mitigate security and privacy issues in cloud environments. Through integrating secure task migration and privacy-preserving mechanisms, QHRMOF can augment the trustworthiness and dependability of cloud-based systems. Additional examination of fault tolerance strategies and proactive anomaly detection can enhance the robustness of QHRMOF in critical environments where system reliability is essential. In brief, QHRMOF provides an exceptionally efficient and scalable approach for task scheduling and resource optimization in cloud environments. The current implementation has demonstrated remarkable performance; however, future enhancements in workload prediction, adaptive learning, real-time deployment, and integration with other cloud platforms will further solidify QHRMOF as a leading solution for dynamic cloud environments.

Acknowledgements

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2025R140), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Authors' contributions

Author Contributions: Umesh Kumar Lilhore, Scaria Alex, Vince Paul, Rahoof P P, Sultan M Aldossary, Sarita Simaiya, Ehab Seif Ghith, Kashif Saleem, and Afraz Hussain Majeed have collectively contributed to this work, bringing expertise from diverse domains. Umesh Kumar Lilhore and Sarita Simaiya provided foundational insights and oversight in structuring the study. Scaria Alex and Vince Paul contributed to the conceptual framework and technical methodologies. Rahoof P P and Sultan M Aldossary played a crucial role in data analysis and interpretation. Ehab Seif Ghith and Kashif Saleem offered valuable perspectives on computational modelling and implementation. Afraz Hussain Majeed enriched the work with industry insights and applications, ensuring practical relevance. Their combined efforts have resulted in a comprehensive and interdisciplinary approach to the research.

Funding

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2025R140), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Data availability

Dataset Availability: The datasets used and analyzed during the study are available upon reasonable request from the corresponding author.

Declarations

Consent for publication

All authors have reviewed and approved the final version of this manuscript for publication.

Consent for publication

This study does not involve human participants, and therefore, no ethical approval was required.

Competing interests

The authors declare no competing interests.

Author details

¹Department of Computer Science and Engineering, Galgotias University, Greater Noida, UP, India

²IT Department, Amal Jyothi College of Engineering, Koovappally P.O., Kottayam Dt., Kerala PIN – 686518, India

³Christ College of Engineering, Irinjalakuda, Thrissur, Kerala, India

⁴Malabar Institute of Technology (MIT), Anjarakandy Integrated Campus, Kannur 670612, Kerala, India

⁵Department of Computer Engineering and Information, College of Engineering in Wadi Alddawasir, Prince Sattam University, Wadi Ad-Dawasir, Saudi Arabia

⁶Department of Mechatronics, Faculty of Engineering, Ain Shams University, Cairo 11566, Egypt

⁷Department of Electrical Engineering, College of Engineering, Princess Nourah bint Abdulrahman University, P.O. Box 84428, Riyadh 11671, Saudi Arabia

⁸Department of Research, Arba Minch University, Arba Minch, Ethiopia

Received: 28 March 2025 / Accepted: 21 August 2025

Published online: 02 October 2025

References

1. Naik B, Balaji (2025) Bollu priyanka, and Sarfaraj Alam ansari. Energy-efficient task offloading and efficient resource allocation for edge computing: a quantum-inspired particle swarm optimization approach. *Cluster Comput* 28(3):155
2. Mohammad H, Zade, Behnam N, Mansouri, Mohammad Masoud Javidi (2025) An efficient task scheduling algorithm leveraging fuzzy quantum atom search optimizer with innovative migration for cloud computing. *Cluster Comput* 28(4):269
3. Priyanka B, Naik BB (2025) Energy and cost aware workflow offloading using quantum inspired differential evolution in the cloud environments. *J Netw Syst Manage* 33(1):19
4. Hussain M, Wei L-F, Rehman A, Ali M, Waqas SM, Abbas F (2024) Cost-aware quantum-inspired genetic algorithm for workflow scheduling in hybrid clouds. *J Parallel Distrib Comput* 191:104920
5. Li P, Wang H (2024) Towards sustainable cloud computing: load balancing with Nature-Inspired Meta-Heuristic algorithms. *Electronics* 13:13
6. Veeramachaneni V (2025) *J Netw Secur Data Min ptimizing Renewable Energy Integration in AI-Driven Data Centers Using Quantum Algorithms* 8(1):36-480
7. Komarasamy D, Ramagathan SM, Kandaswamy DM, Gokuldhhev Mony (2025) Deep learning and optimization enabled multi-objective for task scheduling in cloud computing. *Network: Comput Neural Syst* 36(1):79–108
8. Bandyopadhyay B, Kuila P, Bey M, Govil MC (2024) Quantum-Inspired Differential Evolution for Freshness-Aware Caching-Aided Offloading in Digital

- Twin-Enabled Internet of Vehicles. *IEEE Trans Intell Vehicles* 9(12):7646–7659. <https://doi.org/10.1109/TIV.2024.3401033>
9. Chandrasiri S (2025) Energy-Efficient dynamic workflow scheduling in cloud environments using deep learning. *Sensors* 25(5):1428
 10. Rostami S, Ali Broumandnia, and, Khademzadeh A (2024) An energy-efficient task scheduling method for heterogeneous cloud computing systems using capuchin search and inverted ant colony optimization algorithm. *J Supercomputing* 80(6):7812–7848
 11. Brahman MG, VAR (2024) VMMISD: An Efficient Load Balancing Model for Virtual Machine Migrations via Fused Metaheuristics With Iterative Security Measures and Deep Learning Optimizations. *IEEE Access* 12:39351–39374. <https://doi.org/10.1109/ACCESS.2024.3373465>
 12. Acharya B, Panda S, Das S et al (2025) Optimizing task scheduling in cloud environments: a hybrid golden search whale optimization algorithm approach. *Neural Comput Appl* 37:10851–10873. <https://doi.org/10.1007/s00521-025-11113-9>
 13. Chauhan J, Taj Alam (2024) Adjustable rotation gate based quantum evolutionary algorithm for energy optimization in cloud computing systems. *Int J Comput Sci Eng* 27(4):414–433
 14. Mahapatra A, Mishra K (2024) Rosy pradhan, and Santosh Kumar majhi. Next generation task offloading techniques in evolving computing paradigms: comparative analysis, current challenges, and future research perspectives. *Arch Comput Methods Eng* 31(3):1405–1474
 15. Hu W, Cao Q, Darbandi M, Nima Jafari Navimipour (2024) A deep analysis of nature-inspired and meta-heuristic algorithms for designing intrusion detection systems in cloud/edge and iot: state-of-the-art techniques, challenges, and future directions. *Cluster Comput* 27(7):8789–8815
 16. Pandi M, Senthil Kumar A (2025) Efficient workload allocation in IoT Fog-Cloud architectures for energy efficiency and low latency using DEABC algorithm. *KSII Trans Internet Inform Syst* 19(2):368. <https://doi.org/10.3837/tii.s.2025.02.002>
 17. Jwala GNSSL, Venkata P, Pooja NS, Sharon SR, Sulthana (2024) Suresh. An OptiAssign-PSO based optimisation for Multi-Objective Multi-Level Multi-Task scheduling in cloud computing environment. *i-manager's J Cloud Comput* 11(1):1
 18. Sharma S, Pradeep Singh R (2024) Efficient resource allocation in cloud environment using SHO-ANN-based hybrid approach. *Sustainable Oper Computers* 5:141–155
 19. Recalde A, Cajo R, Velasquez W, Manuel S, Alvarez-Alvarado (2024) Machine learning and optimization in energy management systems for plug-in hybrid electric vehicles: a comprehensive review. *Energies* 17:13
 20. Bapuram B, Subramanian M, Mahendran A, Ghafir I, Ellappan V, Hamada M (2024) Extended water wave optimization (EWWO) technique: a proposed approach for task scheduling in IoMT and healthcare applications. *Evol Intel* 17(5):3609–3620
 21. Bhasker B, Kaliraj S, Gobinath C, Sivakumar V (2025) Optimizing energy task offloading technique using IoMT cloud in healthcare applications. *J Cloud Comput* 14(1):9
 22. Sudheer Mangalampalli S, Reddy Karri G, Nandan Mohanty S, Ali S, Alamri AM, Alqahtani SA (2024) Efficient Hybrid DDPG Task Scheduler for HPC and HTC in Cloud Environment. *IEEE Access* 12:108897–108920. <https://doi.org/10.1109/ACCESS.2024.3435914>
 23. Satouf A, Hamidoğlu A, Gül ÖM, Kuusik A, Durak Ata L, Kadry S (2025) Metaheuristic-based task scheduling for latency-sensitive IoT applications in edge computing. *Cluster Comput* 28(2):1–17
 24. Zhou P, Cao Q, Cao B, Chen W, Jin B, Zhao F (2025) CECHO-GCS: A New Green Energy-Efficient Clustering Protocol Based on Intelligent Optimization Theory in Industrial IoT. *IEEE Internet Things J* 12(8):10907–10919. <https://doi.org/10.1109/IJOT.2024.3514301>
 25. Hussien AG, Chhabra A, Hashim FA, Pop A (2024) A novel hybrid artificial Gorilla troops optimizer with honey Badger algorithm for solving cloud scheduling problem. *Cluster Comput* 27(9):13093–13128
 26. Yanamala AK (2024) Emerging challenges in cloud computing security: A comprehensive review. *Int J Adv Eng Technol Innovations* 1(4):448–479
 27. Zhang M, Yuan B, Li H, Xu K (2024) LLM-Cloud Complete: Leveraging cloud computing for efficient large language model-based code completion. *Journal of Artificial Intelligence General science (JAIGS)* ISSN: 3006–4023 5(1):295–326
 28. Tirlangi S, Teotia S, Padmapriya G, Senthil Kumar S, Dhotre S, Boopathi S (2024) Cloud Computing and Machine Learning in the Green Power Sector: Data Management and Analysis for Sustainable Energy. In: Sharma S, Prakash A, Sugumaran V (eds) *Developments Towards Next Generation Intelligent Systems for Sustainable Development*. IGI Global Scientific Publishing, p 148–179. <https://doi.org/10.4018/979-8-3693-5643-2.ch006>
 29. Mikram H, Kafhali SE, Saadi Y (2024) HEPGA: A new effective hybrid algorithm for scientific workflow scheduling in cloud computing environment. *Simul Model Pract Theory* 130:102864
 30. NASA iPSC Workload Dataset; access on 10th (2024) available at https://www.cs.huji.ac.il/labs/parallel/workload/L_nasa_ipsc/
 31. HPC2N Workload Dataset (2024) access on 10th Feb online available at https://www.cs.huji.ac.il/labs/parallel/workload/L_hpc2n/index.html
 32. Google Cluster Workload Dataset, access on 12th (2024) Feb online available at <https://github.com/google/cluster-data>
 33. Namasudra S, Roy P (2018) PpBAC: popularity based access control model for cloud computing. *J Organizational End User Comput (JOEUC)* 30(4):14–31
 34. Balasubramaniam S, Vijesh Joe C, Sivakumar TA, Prasanth A, Satheesh Kumar K, Kavitha V, Rajesh Kumar Dhanaraj (2023) Optimization enabled deep learning-based ddos attack detection in cloud computing. *Int J Intell Syst* 2023(1):2039217. <https://doi.org/10.1155/2023/2039217>
 35. Lilhore U, Kumar S, Simaiya YN, Prajapati AK, Rai ES, Ghith M, Tlija T, Lamoudan, Abdelaziz A (2025) Abdelhamid. A multi-objective approach to load balancing in cloud environments integrating ACO and WWO techniques. *Sci Rep* 15(1):12036
 36. Li H, Zhang X, Li H, Duan X, Xu C (2024) SLA-based task offloading for energy consumption constrained workflows in fog computing. *Future Generation Comput Syst* 156:64–76
 37. Li H, Liu L, Duan X, Li H, Zheng P, Tang L (2024) Energy-efficient offloading based on hybrid bio-inspired algorithm for edge–cloud integrated computation. *Sustainable Comput Inform Syst* 42:100972. ISSN 2210-5379. <https://doi.org/10.1016/j.suscom.2024.100972>
 38. Li H, Zheng P, Wang T, Wang J, Liu T (2023) A multi-objective task offloading based on BBO algorithm under deadline constrain in mobile edge computing. *Cluster Comput* 26(6):4051–4067
 39. Nagalakshmi B (2025) Multi-objective energy aware task scheduling using orthogonal learning particle swarm optimization on cloud environment. *Int J Inform Technol* 17(1):447–454
 40. Fang Z, Yu H, Fan G, Li Z, Zhang J (2025) Energy, Cost and Reliability-Aware Workflow Scheduling on Multi-Cloud Systems: A Multi-Objective Evolutionary Approach. *IEEE Trans Netw Serv Manage*. <https://doi.org/10.1109/TNSM.2025.3587108>
 41. Singh G, Amit K (2024) Chaturvedi. A cost, time, energy-aware workflow scheduling using adaptive PSO algorithm in a cloud–fog environment. *Computing* 106(10):3279–3308
 42. Lilhore U, Kumar S, Simaiya YK, Sharma A, Kumar Rai SM, Padmaja KV, Nabila V, Kumar (2025) Roobaea alroobaea, and Hamed alsufyani. Cloud-edge hybrid deep learning framework for scalable IoT resource optimization. *J Cloud Comput* 14(1):5
 43. Devi N, Dalal S, Solanki K, Dalal S, Lilhore UK (2024) Sarita simaiya, and Nasratullah nuristani. A systematic literature review for load balancing and task scheduling techniques in cloud computing. *Artif Intell Rev* 57(10):276
 44. Lilhore U, Kumar S, Simaiya KBV, Brahma Rao VVR, Maheswara Rao YK, Sharma R, Alroobaea H, Alsufyani (2025) Majed alsafyani, and MD monish khan. Hybrid DRL-Enhanced ACO-WWO for efficient resource allocation and Load-Balancing in cloud computing. *Int J Comput Intell Syst* 18(1):148

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.